LIÈGE université
Sciences Appliquées

Major project in electrical engineering

CAMILLE

Cocktail machine

Authors:
Maud Casaril s202160
Pierre Dalem s191571
Arnaud Duvivier s203457
Florentin Godfirnon s201328
Melissa Kaci s193784
Julien Vanderheyden s203734

Professors:
M. Biron
B. Boigelot
G. Drion
JM. Redouté

2023-2024

Acknowledgements

Throughout this project we received tremendous help and priceless advice from many people. We would like to express our sincere appreciation to the following individuals:

- Mr. Marc Biron for all the help regarding the management of our project,
- Pr. Bernard Boigelot, Pr. Guillaume Drion and Pr. Jean-Michel Redouté for answering all our questions and concerns,
- Morgane Diepart and Thibault Gillis for all the precious advice and feedback,
- Thomas Pirottin for the sharing valuable knowledge and a brilliant last minute solution,
- Angel Calderon Jimenez for all the technical help and tips during the soldering of or PCB,
- Thérèse and Jean-Phillipe for all the wood-working help,
- All the Fab52 personnel for the countless 3D parts and laser cutting,
- AEES for the continued support and extra funding,

Without their unwavering support this project would not have been possible.

Abstract

Who hasn't waited 30 minutes in line at a bar to get a cocktail? Employing more bartenders? It may be a short-term solution, but it's not the best one, given the limited space behind the bar. But new technologies are enabling us to find an even more optimal solution, practical for bartenders, barmaids, bar owners and, above all, for customers. It's called *Camille*, and it's ready to revolutionize the bar as we know it today.

This machine will enable bartenders to automate the creation of cocktails, without replacing them, and thus enable the bar to perform better and customers to wait less.

Contents

1 Introduction **1**

2 Hardware **2**
2.1 Servo motors . . . 2
2.2 Water level sensor . . . 3
2.3 Stepper Motor . . . 5
2.4 Stop switch . . . 6
2.5 DC Pump . . . 7
2.6 Weight sensor . . . 8
2.7 Power supply . . . 10

3 Software **11**
3.1 Algorithmics . . . 11
3.1.1 Software architecture . . . 11
3.1.2 Buttons . . . 13
3.1.3 Stepper motor . . . 13
3.1.4 Servo motor . . . 14
3.1.5 Controller . . . 14
3.2 Peripherals . . . 15
3.2.1 CCP Module . . . 15
3.2.2 Interrupt on Change (IoC) . . . 16
3.2.3 Schmitt Trigger . . . 16
3.2.4 SSP Module . . . 16
3.2.5 Indirect Memory Adressing . . . 18

4 Printed Circuit Board (PCB) **19**
4.1 Schematic . . . 19
4.2 PCB Layout . . . 20
4.3 Soldering . . . 21

5 Control System **23**
5.1 Goal of the controller . . . 23
5.2 Input of the controller . . . 23
5.3 Output of the controller . . . 25
5.4 Controller . . . 25
5.4.1 Choice of the values K_p . . . 26
5.5 Other ideas of controller . . . 26
5.5.1 Controller with a table of conversion . . . 26
5.5.2 Controller in two phases . . . 27

6 Mechanics **29**
6.1 Wooden structure . . . 29
6.1.1 Main frame . . . 29
6.1.2 Laser cut details . . . 30
6.2 Sliding Platform . . . 31
6.2.1 3D printed details . . . 32
6.3 Relation between the dispensers and the servos . . . 33
6.4 PCB placement and cable routing . . . 34

7 Management and risk assessment **37**
7.1 Project Charter . . . 37
7.2 Planning . . . 37
7.3 Acceptance Criteria . . . 38
7.4 Risks assessment . . . 38
7.5 Budget management . . . 40

8 Improvements **41**
8.1 What could have been done differently . . . 41
8.2 Further improvements . . . 41

9 Conclusion **42**

A Appendix **44**
A.1 Component data-sheets: . . . 44
A.2 PCB components data-sheets: . . . 44
A.3 Codes: . . . 44
A.4 Detail planning . . . 44
A.5 Video . . . 44
A.6 Renderings of 3D parts . . . 44
A.6.1 Renders from section 6.2 . . . 44
A.6.2 Renders from section 6.3 . . . 45

1 Introduction

This project is part of the *Major Project In Electrical Engineering [APRI0007]* course and was developed by six students in the master of electrical engineering program at the University of Liège.

The goal of this project is to build a machine that automatically prepares cocktails. To do so, the machine, named *Camille*, is composed of a rail that supports a moving platform and a structure that allows various fluid dispensing mechanisms to be mounted above the height of the glass, as can be seen in Figure 2. The moving platform slides across the rail with the help of a stepper motor and supports the glass as well as a weight sensor. This assembly enables the platform to be positioned under the appropriate fluid dispenser. To serve the alcohol, three pods containing different beverages are used to create cocktails. A servo motor per dispenser allows the alcohol to flow from the pod to the glass. To choose the desired cocktail, a **Selection** and a **Validation** button are used. The first is used to select one of three possible alcohols: Ricard, fruity Peterman, or Get 27. Once the desired alcohol is selected, the **Validation** button must be pressed. The second phase of the selection process then begins, consisting of choosing whether the user wants water in the glass or just a shot of alcohol. After this stage, the choice must be validated once again with the **Validation** button. Once the drink is selected, the machine operates and the cocktail is then ready to be enjoyed!

At the start of the project, a schematic representing the overall idea of the machine was drawn up, as shown in Figure 1. It can be seen that this initial concept is very close to the final machine, which is depicted in Figure 2.

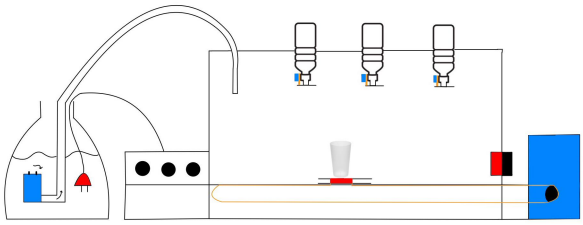


Figure 1: First schematic of the machine

Figure 2: Photo of the completed cocktail machine

2 Hardware

In this section, the specifics of each electrical component will be detailed, along with their functions and limitations.

2.1 Servo motors

The servo motors are used to linearly activate the alcohol dispensers. The rotational motion of the servo is transferred vertically using a combination of 3D-printed parts, as explained in detail in Section 6.3. These particular servos were selected because they are high-torque models capable of producing a maximum force of up to 11 kg/cm when using the maximum voltage of 6V. The voltage range for these components is from 4.8V to 6V. Since this application does not require the maximum torque capacity of the servos, they are supplied with 5V. This voltage has proven to provide sufficient torque to effectively activate the alcohol dispensers.

The specifications permit a rotation of each servo of up to 120°, ranging from -60° to 60°. However, for this particular application, only an 80° rotation is required. This is achieved by varying the duty cycle while maintaining a constant pulse period. The servo operates with a 20 ms pulse width. According to the datasheet, the duty cycle is divided as follows (illustrated in Figure 3):

- **1ms**: is the minimum duty cycle we must send to the servo and represents ***+60°***;
- **1.5ms**: is the ***0°*** position;
- **2ms**: 2ms is the maximum duty cycle and represents ***-60°***.

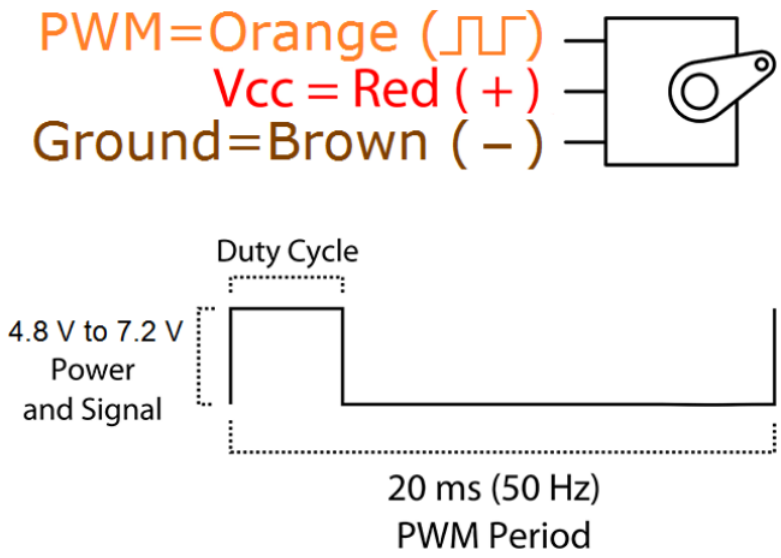


Figure 3: Illustration of the servo motor's duty cycle and PWM along with the pin assignment

The specific duty cycle used to achieve an 80° rotation involves varying between a low and a high position. These positions depend on the orientation of the 3D-printed arm attached to the servo. In this application, the low position chosen corresponds to -60° of the servos, with a high voltage pulse of 2 ms, and the high position is +30°, which corresponds to a high voltage pulse of 1.25 ms.

As shown in Figure 3, each servo is equipped with three pins. The entire cocktail creation process is sequential, meaning only one servo is activated at any given time. Consequently, all three motors share a common VCC and ground connector on our PCB.

Additionally, there is a three-pin connector that comprises the three PWM pins of each servo (refer to Section 4). In practice, these three PWM pins are controlled by a single PWM pin on the PIC16. This PWM pin feeds three "AND gates" and three output pins of this AND gate micro-controller are used to select the desired PWM output pin for the servos.

2.2 Water level sensor

The water level sensor is used inside the large water container to determine if the remaining volume of fluid is sufficient to fill a cocktail. The output signal of the sensor changes depending on whether the sensor is immersed or not. This change in output signal is triggered by a variation in the resistance value between the two electrodes of the sensor.

Indeed, as water is a much better conductor than air, the resistance between the two electrodes decreases when the sensor is immersed, which, in turn, reduces the total resistance of the sensor. The opposite behavior is observed when the sensor is removed from the water.

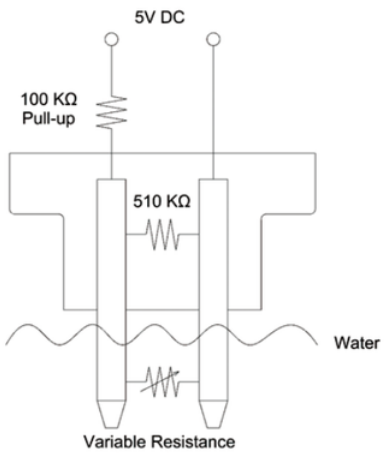


Figure 4: Diagram of the water level sensor

As shown in the diagram of the sensor in Figure 4, the sensor includes a built-in 510kΩ resistor and operates in a pull-up configuration. The variable resistance between the two electrodes, which are situated in water, facilitates the detection of immersed and dry states. In this case, a 68kΩ pull-up resistor was used instead of the standard 100kΩ to allow for a greater range between the two states.

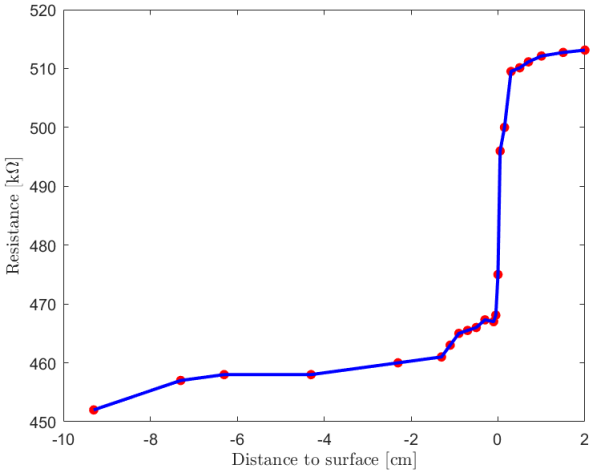


Figure 5: Evolution of the resistance in the electrodes of the sensor as a function of their height with respect to the surface of the water

The graph in Figure 5 plots the evolution of the resistance across the electrodes of the sensor as a function of their height relative to the water's surface. Two distinct regimes can be identified: immersed and out of the water.

Below the surface, the resistance measures approximately 460kΩ. This occurs because water is conductive, causing the sensor to behave as if two resistances were connected in parallel. A quick calculation can determine the resistivity of the water over the 1cm distance between the two electrodes:

$$\frac{1}{R_T} = \frac{1}{R_1} + \frac{1}{R_2}$$

$$R_2 = \frac{R_1 \cdot R_T}{R_1 - R_T} = \frac{510k\Omega \cdot 460k\Omega}{510k\Omega - 460k\Omega} = 4692k\Omega \simeq 4600k\Omega$$

The second regime occurs when the sensor is no longer in contact with water. In this case, the electrodes are exposed only to air, with a resistance in the range of $[1.5 \times 10^{13}\Omega; 6 \times 10^{13}\Omega]$. This resistance is practically similar to an infinite resistance, meaning no current will flow through the electrodes. Consequently, the total resistance of the sensor remains at 510kΩ.

A slight bulge observed in the graph (Figure 4) just before the water surface (at 0cm) could be attributed to surface tension. This causes some liquid to cling to the electrodes and slightly warps the surface of the water. Once the surface tension breaks, the surface returns to a perfectly horizontal state.

The sensor is positioned 4mm above the bottom of the tank, which measures 26x16.5 cm. Therefore, when the sensor is triggered, there is approximately 170ml of fluid remaining. This volume is indeed insufficient to fill a vacant 200ml 'Ecocup' after dispensing the alcohol.

2.3 Stepper Motor

The datasheet of the pins was found on the internet and given as follows:

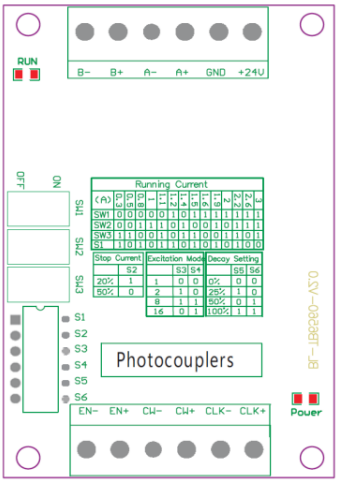


Figure 6: Stepper motor driver pins

Operating the stepper motor is quite straightforward: we simply need to send a PWM signal to the CLK+ pin. Depending on the driver's settings, each 1 to 16 pulses on the CLK+ pin causes the stepper motor to rotate one step. The period of this PWM signal determines the speed of the stepper motor. We have extensively tested different periods of the PWM signal and observed that the current drawn by the motor varies significantly with its speed. Poorly parameterized speeds cause the motor to draw more current and overheat, which is detrimental for long-term applications. Therefore, careful consideration of the motor speed is crucial. In this project, we opted for a speed of one step every 2 milliseconds, which balances the current draw and heat dissipation efficiently. Maintaining this period range is essential.

The advantage of using this driver is that it eliminates the need for us to manage the current supplied to each coil of the stepper motor; the driver handles this automatically. Additionally, we can directly connect the driver's power supply to our 12V supply. The I/O pins from the micro-controller can also be connected to the driver without worrying about specific grounding requirements. This setup allows for the use of separate power supplies for the micro-controller and the stepper motor.

The connections between the stepper and the PIC are established by adding a 2-pin connector for the power supply and another 3-pin connector. For the power supply connector, one pin is connected to the 12V power supply, and the other is connected to a transistor. This transistor is intended to link the ground to the stepper, with its gate voltage connected to an output pin of the PIC. Initially, this configuration allowed the RD1 pin to control the stepper motor's rotation. However, after designing the PCB, it became apparent that the third pin of the driver (the EN+ pin) was a better option for controlling the stepper. Indeed, setting its value to a logic 1 completely disables any supply and movement of the stepper, which was precisely what we needed. Additionally, because the stepper requires a significant amount of current and depends on the speed of the driver, it is necessary to add decoupling capacitors near the power pins of the driver to reduce noise on the power supply. The second connector has three pins: one is connected to the ground, another sends the PWM signal to rotate the stepper, and the last one controls the direction of rotation.

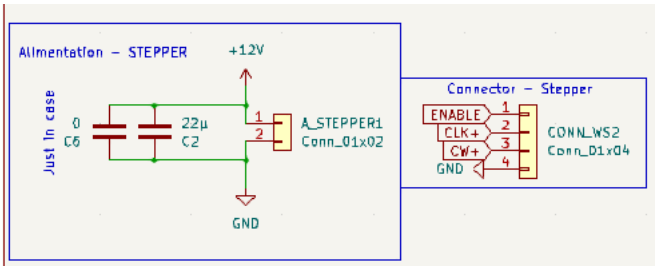


Figure 7: Connections between the connector to the stepper motor and the PIC

2.4 Stop switch

The stop switch is a simple mechanical switch. It is used to reset the position of the moving platform upon startup of the machine or after the creation of a cocktail. Additionally, it serves as a safety mechanism to prevent drift of the stepper's position, such as when the belt slips a few notches. The internal mechanism of the switch is illustrated in Figure 8.

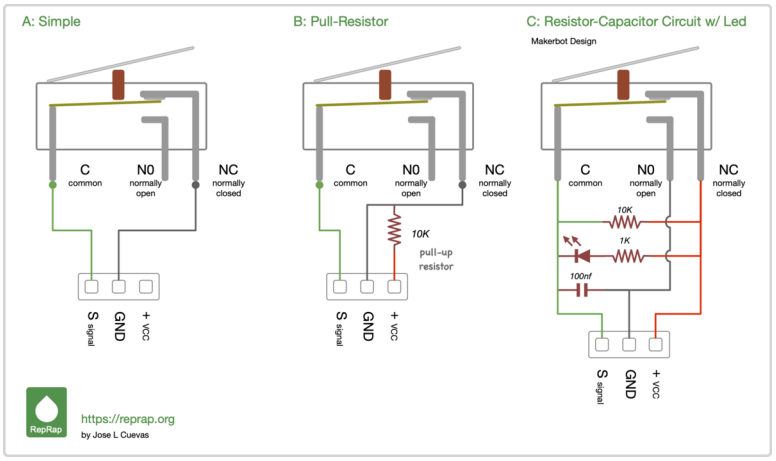


Figure 8: Diagram of the 3 possible configurations for the stop switch

There are 3 possible configurations for the stop switch. To have a clean signal, configuration B (fig 8) was chosen. Although resistors and capacitors could be soldered directly onto the switch, as there is space provided on the switch's PCB for this purpose as can be seen in Figure 9, the components were instead placed on the main PCB. Their placement is detailed in Section 4.

Figure 9: Stop switch seen from above

2.5 DC Pump

The DC pump is used to pump water from the tank, which is the final step in cocktail creation. It is designed to pump approximately 200ml of liquid to complete the drink after adding a dose of alcohol. A pump was chosen over a fixed-volume dispenser as the total amount of liquid to be delivered is greater. Specific pumps that can dispense drinkable fluids without direct contact between the liquid and the pump's interior were considered but they proved to be expensive and slow.

Initially, a 3.3V DC pump was used, but this presented two issues. The first was the need for a second LDO to provide 3.3V on the PCB exclusively for the pump. The second issue was that the pump lacked sufficient power to move the liquid from the base of the cocktail machine to the top. Consequently, it was decided to replace the pump with a more powerful 5V pump, eliminating the need for the second LDO.

The characteristics of the upgraded pump are as follows: it requires a 5V voltage supply. The flow rate of the pump can vary between 1.2 and 2 L/min, which is controlled by varying the duty cycle of the pulse width modulation. Along with data from the weight sensor, this flow rate is utilized in the control loop, with a detailed explanation provided in Section 5.

When integrating the pump with the rest of the circuit, a significant problem occurred: the pump caused the micro-controller to reset for an unclear reason. As illustrated in Figure 10 (where the PWM signal is shown in yellow and the current in the pump in blue), certain currents appeared to be too large, triggering a reset and stopping the PWM signal. After extensive debugging and discussions with the assistants, it was discovered that a diode was missing. Since the DC pump is an inductive load, it requires a freewheeling diode to allow the inductance to discharge when the voltage is turned off by the PWM signal. This behavior can be observed in Figure 11, where the PWM is plotted in yellow and the voltage at the output of the diode in blue. The crucial zone is the "high" part of the blue trace: the voltage slightly exceeds 5V, representing the 5V from the power supply plus the 0.7V drop across the diode. This zone indicates when the pump discharges into the diode.

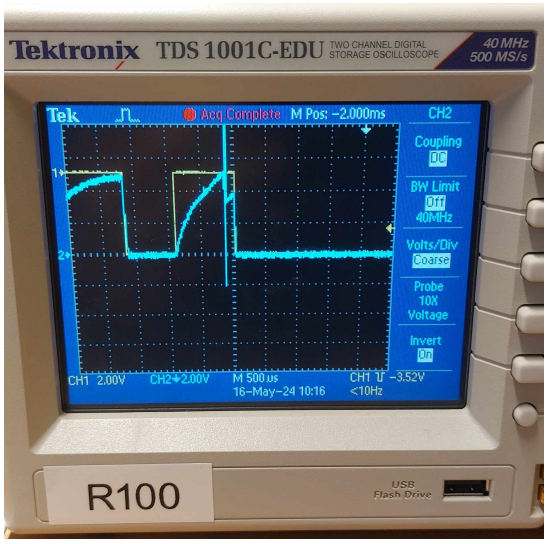


Figure 10: Oscilloscope Traces of the "bug" making the PIC16 reset

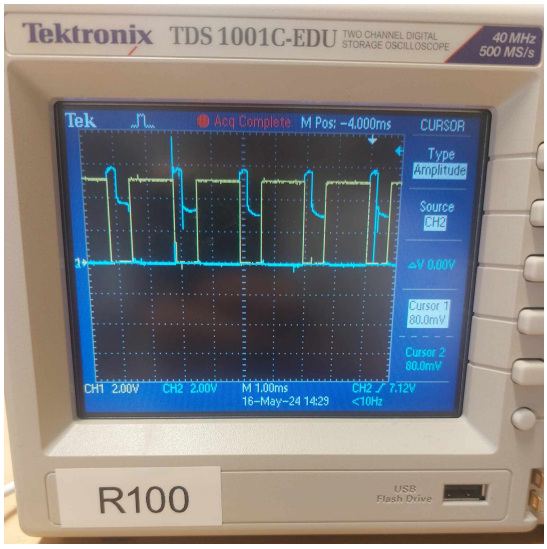


Figure 11: DC pump waveform when a freewheeling diode has been added

In the project, PWM using a CCP module has been used to supply the DC pump and set its power to a desired value. The use of the CCP module will be explained in section 3.2.1 but what is of interest is that the frequency of the PWM is around 500 Hz.

2.6 Weight sensor

To facilitate monitoring of the glass's filling level, a weight sensor is discreetly integrated within the sliding box. This sensor operates on a load cell mechanism, which measures changes in resistivity within a metallic component sandwiched between two plexiglass plates, as illustrated in Figure 12. The resistivity measurements are captured by the HX711 chip, a precision 24-bit analog-to-digital converter (ADC) specifically designed for weight scales. These measurements are processed and stored by an onboard MCU, which provides various functionalities, including a built-in calibration algorithm. Crucially for this project, the I2C communication feature is pivotal, enabling seamless interaction between the sensor and the PIC16F microcontroller via the SSP module (refer to Section 3.2.4).

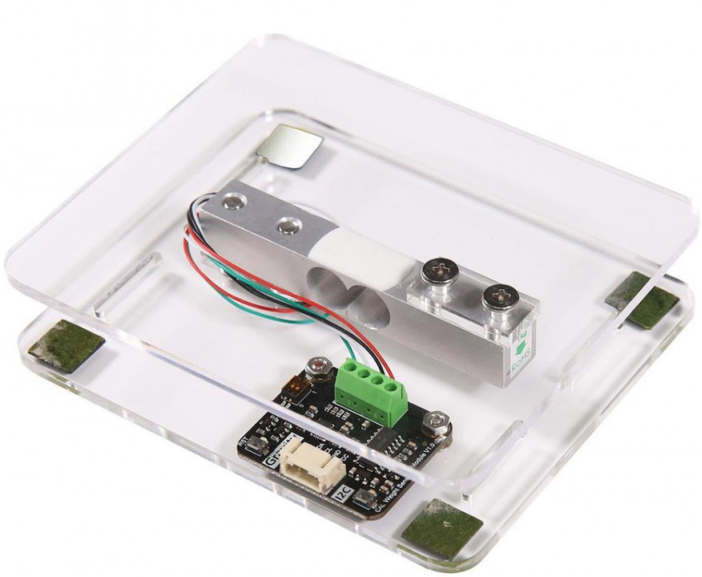


Figure 12: Weight sensor when all parts are assembled

The device can be supplied with a voltage between 3.3V and 5V; thus, the 5V voltage has been chosen since it is already available on the PCB. The measuring range extends up to 1kg, which is more than sufficient for this application. When the HX711 outputs a maximal value (0xFFFFFF), the corresponding weight is 2.2kg (though it is not advised to reach this point as the datasheet only guarantees proper functioning up to 1kg). Consequently, the resolution of the device is calculated as $\frac{max_{weight}}{2^{24}} = 1.192 \times 10^{-}4g$. However, in practice, when a constant weight was applied to the sensor, the output showed significant fluctuations, leading to an overall accuracy of about 0.5g (more details can be found in Section 5.2), which is acceptable for this application.

In order to reach this accuracy the proper method would be to make several readings and to average them in order to smooth the outputs. Even if the division operation is really hard to implement in assembly language, the idea was to make a number of readings corresponding to a power of two so that the final value can simply be shifted right. But, since this still required to temporarily increase the number of bits considered, the system was tested without this smoothing operation and the performances were satisfying. Consequently, this feature is not included in the final implementation.

The primary challenge encountered with the I2C communication was the lack of detailed information in the datasheet, which provided minimal details such as addresses without clarifying their corresponding functions. To overcome this hurdle, we turned to an Arduino library that contained essential functions. By reverse-engineering this library, we gained insights into

the communication operations, including addresses, message sequencing, timing requirements, and calibration procedures. A hyperlink to this library is available in Section A.

Practical implementation of the I2C communication proved far more arduous than anticipated, presenting numerous hurdles from both hardware and software perspectives. Hardware challenges surfaced early, with the onboard MCU becoming unresponsive after a single failed communication attempt, rendering subsequent responses from the sensor inaccessible. Identifying the necessity to reset the sensor after communication errors prolonged troubleshooting efforts.

Additionally, initial discrepancies in timing between the interrogation and response phases posed another obstacle, requiring a comprehensive understanding of their significance in ensuring the seamless operation of the communication protocol.

Furthermore, a significant challenge arose in identifying an initialization phase wherein specific registers needed to be communicated to the sensor to enable proper operation. Lacking documentation, determining the correct sequence proved particularly tedious.

Transitioning from the breadboard to the PCB and integrating the weight sensor into the overall structure introduced further complications. While communication initially functioned intermittently, it eventually ceased altogether after a variable period. Subsequent analysis revealed that this issue stemmed from the increased wire length between the PIC16 and the weight sensor (approximately 60/70 cm compared to 10cm during prototyping). Given I2C's intended use for short-range PCB communication, longer wires rendered the bus susceptible to disturbances, exacerbated by the electromagnetic interference generated by stepper and servo motors. Mitigation measures involved reducing the pull-up resistor values to maintain a stable "HIGH" state on the bus, while ensuring reasonable current levels. Despite these enhancements, occasional corruption of an I2C packet persisted, with no viable software-based solutions identified. Consequently, the bus remained non-responsive until power was cycled and the system reset.

All aforementioned challenges were successfully identified and addressed through vigilant and systematic examination of oscilloscope traces, as illustrated in Figure 13. These traces provided invaluable insights into the intricate workings of the inherently opaque communication protocol, facilitating effective troubleshooting and resolution of issues.

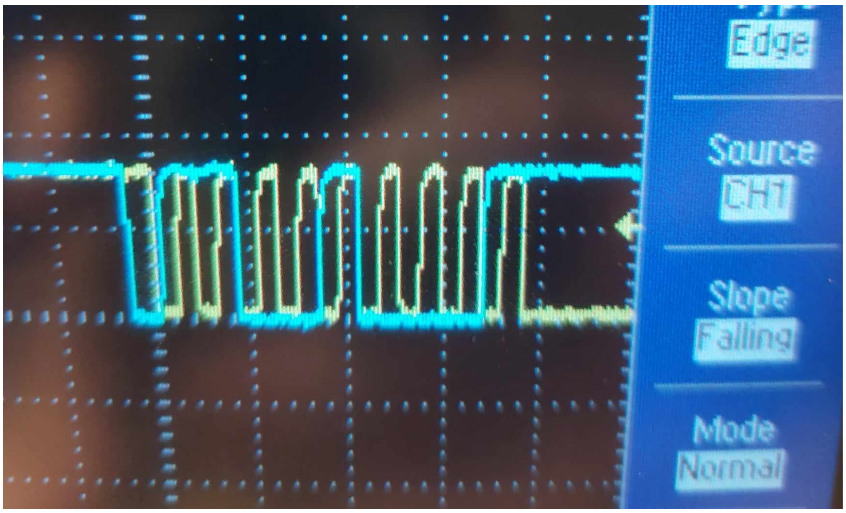


Figure 13: Oscilloscope trace of an I2C communication

2.7 Power supply

To power the entire machine, an AC-DC converter is used. The input is connected to the mains electrical socket, and the output is delivered through a barrel jack to the PCB. This output provides a voltage of 12V with a maximum current of 2.1A. A voltage regulator module (VRM) is then utilized to supply the various hardware components with 5V. Details of this configuration are illustrated in the PCB's layout, which can be found in Section 4.

3 Software

Throughout the progression of this project, we encountered two primary categories of challenges related to the software aspect. Initially, from an algorithmic standpoint, we navigated through challenges ranging from conceptualizing the global architecture of the code to addressing more granular issues such as implementing multiplication operations. The second set of challenges revolved around effectively harnessing the diverse peripherals inherent to the PIC16F micro-controller, crucial for fulfilling the project requirements.

3.1 Algorithmics

3.1.1 Software architecture

After gaining a deeper understanding of the components used and their interactions with the PIC micro-controller, the global structure of the code can be presented. For this purpose a kind of simplified C language is used.

The code is based on the *Round-Robin with interrupts* architecture, which is well suited to our needs since no preemption is required and the non-interrupt tasks can run sequentially. We received feedback on this pseudocode from Mr. Gillis, which allowed us to make modifications and ultimately arrive at the architecture used in this project.

```
//different possible states
typedef enum {
    reset_state,
    selection_state,
    move_to_alcohol_state,
    alcohol_state,
    move_to_pump_state,
    pump_state,
    return_state
} MachineState;
```

Instead of using several flags to enable the different tasks, an enumeration was implemented, as advised by Mr. Gillis. Since the machine involves step by step tasks, the logic is purely sequential, making the machine act as a Finite State Machine. Each state corresponds to a step in the process where something new needs to be launched, done, or checked to proceed with the creation of the cocktail. A set of flags is used by the interrupt routines to signal to the states that certain instructions have finished, allowing the state machine to continue. The only task that operates differently from the state machine is the button check, which is performed periodically thanks to a flag raised by an interrupt routine related to `Timer 0`. This can be implemented with a *switch-case* logic based on this type of enumeration.

```
//Switches have been replaced by a timer
interrupt void timer0_overflow(void){

    //check reset button
    bool reset_button_state = read_button(reset);
    if(previous_reset_button_state == 0 && reset_button_state == 1){
        currentState = reset_state;
    }
    previous_reset_button_state = reset_button_state;

    //check selection button
```

```
bool selection_button_state = read_button(selection);
if(previous_selection_button_state ==  0 && selection_button_state == 1){
    if(currentState = selection_state){
        cocktail = ((cocktail + 1)%4) + 1
        turn_on_leds(cocktail);
    }
}
previous_selection_button_state = selection_button_state;

//check start button
bool start_button_state = read_button(start);
if(previous_start_button_state ==  0 && start_button_state == 1){
    if(currentState == selection_state){
        turn_off_leds();
        currentState = move_to_alcohol_state;
    }
}
previous_start_button_state = start_button_state;
```

In the first version of the pseudocode, it was planned to use the *Interrupt on Change*, so that an interrupt would be triggered each time a digital change appears on one of the specified pins. However, we faced some issues when trying to implement it on the PIC micro-controller and Mr. Boigelot advised us to handle it in an easier way : by having a timer dedicated to this function, periodically checking the state of these sensors, except for the stop switch that uses nevertheless uses IOC. In the Listing 3.1.1 there is a piece of the big interrupt routine that is periodically called to perform the check operation.

```
for(;;){
            switch(currentState) {
            case reset_state:
                send_to_dc_pump(0); //STOP DC PUMP
                if(servo_turned){
                    unturn_servo(cocktail);
                }
                currentState = return_state;
                break;

            case selection_state:
                //LEDs light up according to selection
                initialize_selection_indicators(cocktail);
                break;

            case move_to_alcohol_state:
                //moving the stepper towards the desired alcohol
                move_stepper_to(position, destination  = cocktail);
                if(position == cocktail){
                    currentState = alcohol_state;
                }
                break;
```

However, in the final machine, some particular states do not belong to the sequential processing of the glass creation. First, there are initialization states that are executed only at the startup of the machine. These states are used to move all the servos down and reset the position of the box on the left. Then, there is the reset state, which is activated by pressing the reset button. This state puts the machine in a waiting mode (after everything has been reset) until the user presses the validation button to send the box to the left and start another

cocktail creation. The final state is for the water level sensor, which checks if there is enough water left in the tank. If there is not enough water, the creation process will pause its operation until the water level is sufficient to resume.

Finally, as explained above, the main structure of the code now utilizes a large "switch-case" logic contained within an infinite 'for' loop. Both the interrupts and the logic of the loop are responsible for changing the value of the "currentState" variable, which defines the current phase of execution. This choice is motivated by the fact that our machine does not require tasks to run in parallel. The beginning of this loop can be seen in Listing 3.1.1. The full pseudo-code can be found in appendix A.3 of this report.

3.1.2 Buttons

To select the desired cocktail, two buttons are used: **Selection** and **Validation**. The third button, the **Reset** button, is used to stop the machine. When using the buttons, it is important to avoid bounce, which occurs when a button is pressed. When the button is pressed (or released), the signal changes from high (or low) to low (or high) in a very short time. Ideally, this change would occur instantly, but in practice, a rebound phenomenon causes the signal to oscillate briefly between high and low. As a consequence, if the signal is read during this time, the wrong value can be registered by the machine, leading to operational problems.

Initially, a hardware solution was considered to prevent bounce, but after consulting with Pr. Boigelot, it was decided to shift part of the problem from hardware to software. To address this, a software-based solution was implemented: the button signal is checked twice. After the initial signal check, a delay of 7ms is applied before evaluating the signal again. The values from both checks are then compared. If both values are high, the button state is set to "pressed"; otherwise (if both values are low or the values differ), the button state is set to "not pressed."

3.1.3 Stepper motor

The stepper motor is a motor which works with a PWM signal as input and when a high signal is sent, it will move with steps. To control the position of the platform, the idea is to use a register which contains the current position of the platform and this position will be compared with the value of the target position. All the possible positions (position for the dispensed of soft drink and the 3 positions of the pods for the three different alcohols) are hard-coded as follows and were determined using a trial-and-error method:

```
1 define PUMP_POSITION 32
2 define ALCOHOL_1_POS 113
3 define ALCOHOL_2_POS 180
4 define ALCOHOL_3_POS 250
```

To update the position and give the order of making a movement, a slower clock than the system clock (32MHz) is needed. To do so, the `Timer 1` is used. This timer uses a 16 bit counter and it counts from a starting value (by default 0) to 65535 (`1111 1111 1111 1111b`). The starting value can be configured by setting the timer. In this case, the starting value is set to 57536, which corresponds to `1110 0000 1100 0000b`. This configuration results in the number of counts being

$$65535 - 57536 + 1 = 8000.$$

Given that the instruction clock is used as the source clock, the timer clock frequency is $\frac{F_{osc}}{4} = 8\text{MHz}$, meaning the time between two counts is $\frac{1}{8\text{MHz}} = 0.125\mu s$. To count all the

values from 57536 to 65535, the timer requires $8000 \times 0.125\mu s = 1$ms, giving `Timer 1` a frequency of 1000Hz. Each time the counter overflows, the output value for the servo is inverted, thus creating a PWM signal with a frequency of 500Hz.

This PWM signal is then used to increment a counter that holds a value from 0 to 7. When this counter reaches 7, the actual position of the platform is updated based on its direction, and the new position is compared with the target position to determine if the platform should continue moving. This strategy ensures accurate and real-time information about the platform's position. One of the main risks in the cocktail machine is misplacing the platform that supports the weight sensor, which could lead to damage. To mitigate this risk, each position is hard-coded, and a stop-switch button is present on the left side of the rail. Additionally, the counter's range of 0 to 7 helps reduce the scale of possible positions, allowing them to be represented within 8 bits.

The use of the stop switch also presented challenges in handling the servo movement. It was necessary to address these challenges to ensure the stepper motor could be updated and stopped when the stop switch is pressed.

3.1.4 Servo motor

The signal that controls the stepper motor is a PWM signal, as mentioned before. This PWM signal is generated by the PIC and needs to have a frequency of 50Hz to ensure proper operation, resulting in a period of 20ms. To reduce the frequency of the system clock to 8000Hz (which is necessary for precise duty cycle control), `Timer 1` is used, as it is for the stepper motor. In this case, the target frequency is 8000Hz, meaning the time to count from the starting value to 65535 must be 8 times shorter than in the previous case. To achieve this, the starting value is set to `1111 1000 0011 0000b` (63536), resulting in a count of 1000, which is 8 times smaller than in the previous case.

According to the hardware specifications (see Section 2.1), the low position requires a signal with a duty cycle of 10%. For the high position in our project, the duty cycle needs to be 6.25% to achieve a delay of 1.25ms between the rising and falling edges.

To operate this motor, the duty cycle is adjusted to raise the servo when the corresponding command is received, and then a counter begins to increment. This counter ensures that the servo motor has enough time to completely release all the alcohol from the pods. When this counter reaches the value of `Servo_wait_up`, all the alcohol should be in the 'Ecocup', so the duty cycle changes to its maximum value (10%), and the servo motor moves back to its low position. Again, a counter increments `Servo_wait_down` times the 20ms period before setting a flag to indicate that its operation is finished.

3.1.5 Controller

As will be discussed in Section 5, a P controller was chosen. This means that the controller computes its output based solely on the error, which is the difference between the goal weight and the measured value. This error is then multiplied by a constant K_p.

During assembly implementation, several problems arose. First, the measured weight was represented using 16 bits, while the controller's output needed to be written using 8 bits. To address this issue, only the 8 bits that best represent the weight were retained. This choice is discussed in Section 5, where it was decided to keep the bits from the fifth to the twelfth. To achieve this, the 16-bit measured value is shifted left 4 times to position all the relevant bits within the same register. This adjusted value is then subtracted from the target weight, and

the resulting difference is the error.

After this, the main challenge in the implementation was to multiply the error by the constant K_p. The goal was to multiply an 8-bit number by another 8-bit number. To accomplish this, a review of the "Organisation des ordinateurs" course was necessary to understand how to perform this operation bit by bit. The general idea of this multiplication is to examine each bit of the constant K_p and perform the appropriate operation based on the value of the current bit. If the current bit is 0, nothing is added to the multiplication result, and the bits of the error are shifted to the left. If the current bit is 1, the current value of the error, which may have been previously shifted, is added to the multiplication result, and the bits of the error are shifted to the left. It is crucial to be mindful of the risk of overflow when shifting the bits of the error to the left. Overflow occurs when the left shift of the error or the right shift of K_p produces a one-bit carryover. In such cases, the multiplication result is set to its maximum value, `11111111`, which corresponds to the maximum flux of the DC pump.

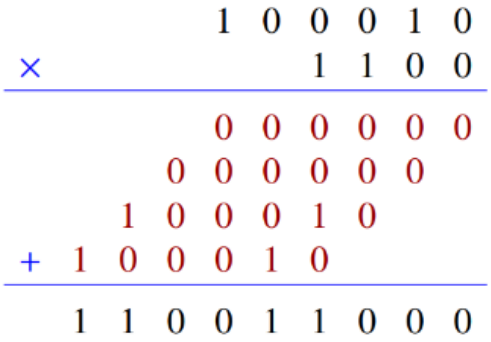


Figure 14: Binary multiplication

3.2 Peripherals

3.2.1 CCP Module

The CCP module is a peripheral that utilizes the resources of Timer 2 for its operation. It can be used in three different modes: Capture, Compare, and PWM. For this project, only the PWM mode is of interest, as it is used to control the power of the DC pump. While the CCP module could also be used for the servo motors, it is not suitable because the minimal frequency of the CCP module with a device frequency of 32MHz is 500Hz, whereas the servo motors require a PWM frequency of 50Hz, making it incompatible.

The operation of the CCP module in PWM mode involves specifying the period by loading a value into the PR2 register using the formula:

$$\text{PWM period} = [(PR2) + 1] * 4 * T_{osc} * \text{TMR2 prescale value}$$

The TMR2 prescale value has been set to 64, and PR2 is set to 255 to achieve the maximum period. The duty cycle can then be set by loading the desired value into the CCPR1L and CCP1CON registers using the formula:

$$\text{Duty Cycle Ratio} = \frac{(CCPR1L : CCP1CON < 5 : 4 >)}{4(PR2 + 1)}$$

In this project, we used the CCP1 module. Each CCP module has two possible output ports. For CCP1, we used RC2 to drive the pump. Special care must be taken to select the PWM mode for CCP1 and then configure the Timer 2 parameters to achieve the desired behavior. Once configured, the PWM module can be enabled as needed.

3.2.2 Interrupt on Change (IoC)

Interrupt-on-change (IOC) is a peripheral available on most of the input pins of the microcontroller. For the pins that support IOC, it is possible to detect a rising or falling edge. Once detected, an interrupt routine specific to each pin is called. However, it is not possible to distinguish between a rising or falling edge, although they can be enabled separately.

The IOC can be enabled by setting the IOCIE bit in the INTCON register. Then, the bit corresponding to the desired pin can be set in the IOCxP register for the rising edge and in the IOCxN register for the falling edge. Once a change is detected, the corresponding bit in the IOCxF register is set to trigger the interrupt routine.

This setup seems straightforward, but initially, it resulted in zero detection of changes. After some investigation and seeking help, it was found that a few additional steps were necessary. First, to correctly detect the value on an input pin, the corresponding bit in the ANSELx register must be explicitly reset to ensure the input pin is in digital mode. Additionally, the weak pull-up must be disabled for the interrupt-on-change to function correctly. Otherwise, the IOC will not work.

3.2.3 Schmitt Trigger

Figure 15 illustrates the behavior of a Schmitt Trigger input. The purpose of this type of input is to avoid oscillations when the input voltage is around the threshold between high and low voltage. This is particularly useful for the water level sensor, which does not have a clear high or low value.

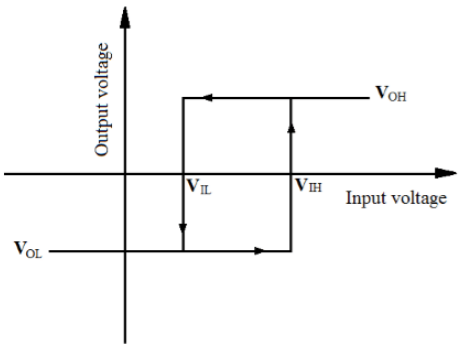


Figure 15: Schmitt Trigger diagram

We chose to connect the inputs of the water level sensor and the stop switch to ensure a clear variation between high and low voltage. The PIC16 has a Schmitt Trigger integrated on most of its pins. To enable the Schmitt Trigger on the desired pin, simply set the corresponding bit in the INLVLx register to one.

3.2.4 SSP Module

The SSP module (Serial Synchronous Port) can be configured to handle either SPI or I2C communication. The I2C interface supports many specific features such as 10-bit addressing, clock stretching, and bus collision detection. However, in this project, the module is kept as simple as possible and is configured in I2C master mode using an internal clock source by setting the appropriate bits in the SSP1CON1 register. The frequency of the I2C clock can be configured by modifying the SSP1ADD register. The corresponding pins (RC3 and RC4) should also be configured as inputs, and the analog inputs (ANSELC) should be disabled.

Once the module is properly configured, interaction with the bus can begin by setting specific bits in the SSP1CON2 and SSP1BUF registers. The SEN bit allows sending a start condition, the PEN bit sends a stop condition, the RSEN bit sends a restart condition, and the ACKEN bit sends either an ACK or NACK depending on the value of ACKDT. Writing a byte to SSP1BUF sends this byte on the bus. Each of these operations is followed by the SSP1IF bit in the PIR1 register being set, indicating that the operation is complete.

The typical sequence of operations is as follows:

1. A Start condition is generated by setting the SEN bit of the SSP1CON2 register.
2. SSP1IF is set by hardware on completion of the Start.
3. SSP1IF is cleared by software.
4. The user writes the SSP1BUF with the slave adress to transmit and the R/W bit set.
5. Address is shifted out the SDA pin until all eight bits are transmitted. Transmission begins as soon as SSP1BUF is written to.
6. The MSSP module shifts in the ACK bit from the slave device and writes its value into the ACKSTAT bit of the SSP1CON2 register.
7. The MSSP module generates an interrupt at the end of the ninth clock cycle by setting the SSP1IF bit.
8. The user sets the RCEN bit of the SSP1CON2 and the master clocks in a byte from the slave.
9. After the 8th falling edge of SCL, SPP1IF and BF (Buffer Full Flag) are set.
10. User clears the SPP1IF bit and reads the received byte from SSP1BUF, which clears the BF flag.
11. The user either clears the SSP1CON2 register's ACKDT bit to receive another byte or sets the ACKDT bit to suppress further data and the initiates the acknowledge sequence by setting the ACKEN bit.
12. Masters's ACK or NACK is clocked out to the slave and SSP1IF is set.
13. User clears SSP1IF.
14. Steps 8-13 are repeated for each received byte from the slave
15. If the ACKDT bit was set in step 11 then the user can send a STOP to release the bus, by setting the PEN bit of SSP1CON2.

Even though the sequence is quite straightforward, its implementation was challenging. The SSP1IF and ACKDT bits must be checked and cleared between each operation, and synchronization with the weight sensor must be perfect. Handling I2C communication became more difficult when interrupts were active, so it was decided to disable interrupts during I2C operations. Considering that the timescale of an I2C communication is on the order of 100µs, this is negligible compared to the timescale of other operations.

3.2.5 Indirect Memory Adressing

Since the weight sensor sends packets of 32 bits and the PIC16 micro-controller has 8-bit memory, it was natural to use arrays. In assembly language, arrays are easy to declare but not so easy to use. To handle this, the easiest method is to use **Indirect Memory Addressing**. Indirect memory addressing allows access to parts of the memory based on the address of the register that needs to be read or written. This is done through the File Select Register (FSR0L and FSR0H), which loads the address of the memory zones to be accessed using the "HIGH" and "LOW" keywords that isolate the 8 upper/lower bits of a register's address. Since memory allocation in the PIC16 is sequential, the elements of an array will have consecutive addresses, and navigating the array is done by incrementing or decrementing FSR0L.

Once the correct address is loaded into the FSR register, you can either write to it or read its content. To write to it, use the "movwi" instruction, which moves the content of the W register into the register pointed to by FSR. To read from it, the content of the memory zone pointed to by FSR is copied at any time into the corresponding INDF register, allowing you to use "movf" to move the content of INDF into the W register.

4 Printed Circuit Board (PCB)

4.1 Schematic

Figure 16 displays the electrical schematic for the project. Some components were added after the PCB order. All the details are explained in section 4.2.

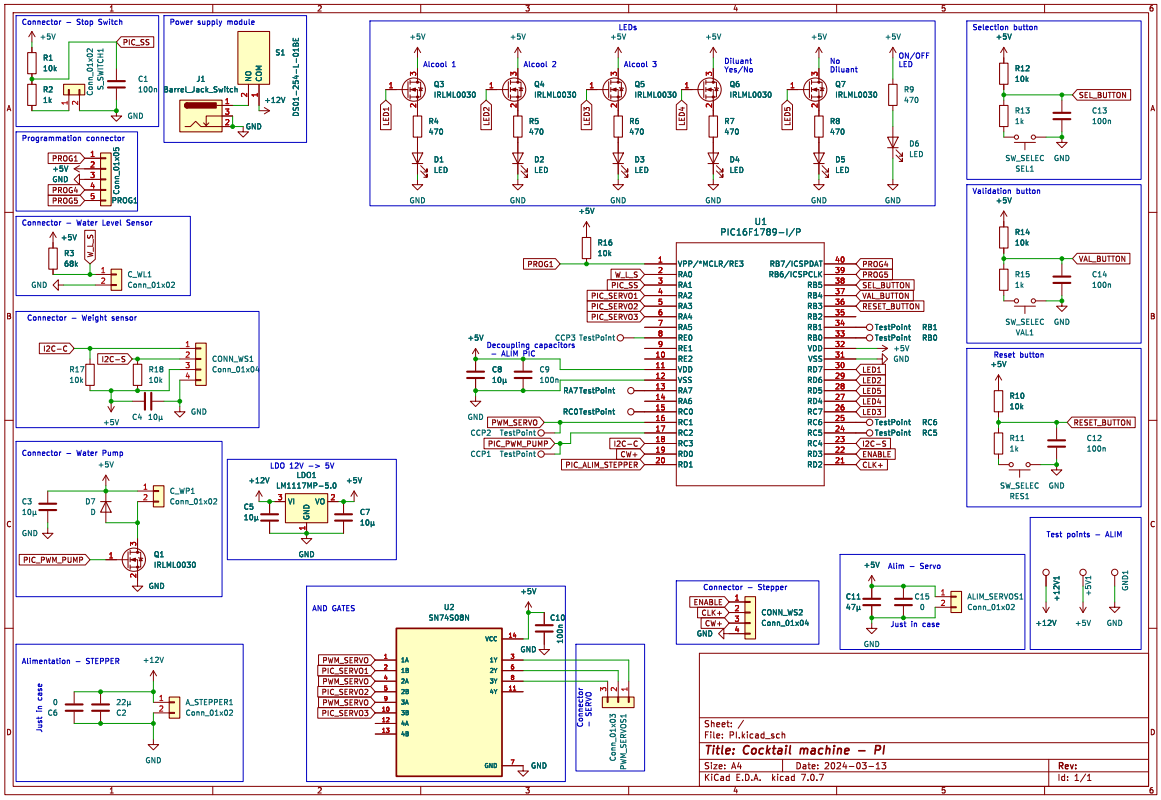


Figure 16: Electrical Schematic

- **Power Supply**: The circuit is powered by a 12V power supply, with a barrel jack connector facilitating a convenient and secure connection point. Decoupling capacitors are placed near critical components to ensure smooth voltage regulation and operation. The 12V supply powers the stepper motor, while a low-dropout (LDO) regulator reduces the voltage to 5V for other components and the microcontroller.
- **Buttons**: Each button is equipped with a capacitor to mitigate signal bouncing, ensuring stable and reliable inputs from the user.
- **Connectors**: All sensors and actuators are connected via JST connectors, facilitating easy assembly, disassembly, and maintenance. The programming connector is different; to interface with the MPLAB PICkit, standard pin headers are used.
- **LEDs**: Transistors are inserted into the circuit before the LEDs, allowing the PIC micro-controller to control them without directly supplying current. This approach provides flexibility and reduces load on the micro-controller. The transistors also protect the micro-controller in case of a short circuit in a branch leading to a pin.
- **AND Gates**: Since not all servos operate simultaneously, a single PWM signal is sufficient. A network of AND gates is used to direct this PWM signal to the correct servo, depending on the application's needs.

- **Test points and pads** To aid in diagnostics, test points are strategically located to measure voltages across the circuit. Additionally, extra pads are provided, allowing for future modifications or additional connections if required.

4.2 PCB Layout

After the initial PCB order, modifications were made without reordering new boards to avoid additional delays. Figure 17 shows the PCB layout before these modifications. The missing parts are highlighted on figure 16. On figure 18, the modifications are shown.

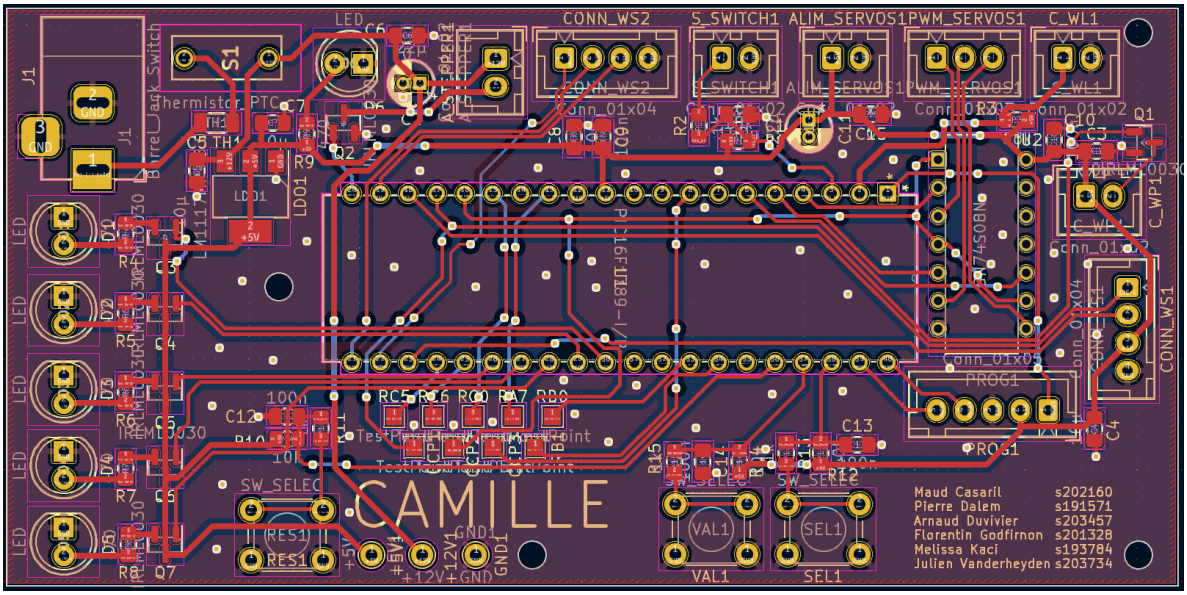


Figure 17: Layout of the Final PCB

The design of the cocktail machine focuses on both user convenience and signal efficiency. Decoupling capacitors are strategically placed as shown in Figure 16, positioned near components where stable voltage is necessary. For aesthetic purposes, SMD components were placed for all the resistors, capacitors and transistors. The idea was to bring out all the important parts and to make all the others "invisible". That is why our LEDs are through hole components and not SMD.

The PCB features ground planes on both the top and bottom layers. This dual-plane design serves several purposes, including reducing ground loops, lowering the impedance of the ground path, and simplifying the PCB layout and routing. By providing a consistent ground reference across the board, the risk of electrical noise and interference is minimized.

To maintain consistent ground potential and avoid voltage differences between the top and bottom planes, vias are used to connect the two ground planes. This ensures a robust and continuous grounding structure, contributing to the reliability and stability of the overall system.

There are three buttons `SEL1` , `VAL1` and `RES1`:
`SEL1` represents the selection button. It is used to navigate for the different choices. The second button, `VAL1` is used to validate the choice. And the last one, `RES1` is used to reset the machine.

The LEDs are strategically placed for optimal visibility. The top LED serves as the "ON/OFF" indicator, lighting up when the device is turned on and remaining off when the device is off. The LEDs on the left side are intended for user interaction. The first three LEDs on the left represent the alcohol selection. Users can cycle through these options using the selection button. Once the desired option is chosen, the validation button confirms the selection.

When this is done, the second stage begins, indicated by the fourth LED turning on. This LED indicates that a non-alcoholic mixer will be added to the drink. If users decide against adding the mixer, they can press the selection button again, which turns off the LED, indicating that the mixer has been removed. Once a choice is confirmed, the validation button is used again to proceed.

The final LED, located at the bottom, acts as a tank-level indicator. It illuminates when the tank is empty and turns off when it is full, providing a clear signal to the user about the device's operational status.

4.3 Soldering

Initially, the surface-mount devices (SMD) components were soldered onto the PCB, followed by the through-hole components. To install the PIC micro-controller, a socket was used instead of direct soldering. This approach was chosen to facilitate debugging: if the PCB had issues or was not functioning properly, the PIC could be easily removed and transferred to a breadboard for testing and debugging. This modular configuration allows for easier troubleshooting and adjustments without compromising the PCB's integrity.

After the test of the PCB, the following issues were identified along with the corrective measures taken:

- The switch selection contained an error: a DIP-switch with a maximum current rating of 25 mA, insufficient for the project's requirements. To correct this, an alternative switch was used, attached with wires on the back of the PCB due to an incompatible footprint. The modification can be seen on the left of figure 18.
- The footprint for the PTC-thermistor was too small, necessitating a short circuit between the pads. This modification removed the protective function of the thermistor. The short circuit was accomplished by soldering a bare wire. It is circled in red in the figure 19.
- Two pull-up resistors were missing from the programming connector for the weight sensor. This was resolved by adding through hole resistors to the back of the PCB. The modification can be seen on the right of the figure 18, two pull-up resistors are connected to a 5V pin and the wanted pin.
- An additional pull-up resistor was required for the PIC's programming connector, which was also added to the back of the PCB. It can be seen on the bottom right of the figure 18.
- The power supply for the PIC was incomplete, with only one side connected instead of both sides as required for a stable 5V supply. The correction involved connecting the power supplies on both sides of the PIC. The existing decoupling capacitor was sufficient. This modification can be seen in the middle of the figure 18.
- A transistor was initially included for the stepper power supply to prevent continuous connection. However, an "enable" pin was found to serve this purpose, so the transistor was short-circuited to prevent potential conflicts. The short circuit is circled in red in the figure 19.
- As explained in the section 2.5, a freewheeling diode was added to the pins of the DC pump connector. It can be seen in black on the bottom right of the figure 19.

These adjustments ensured proper functionality of the PCB, despite the initial errors in layout and component selection. Here on figure 18, a picture of the back of the PCB with all the modifications.

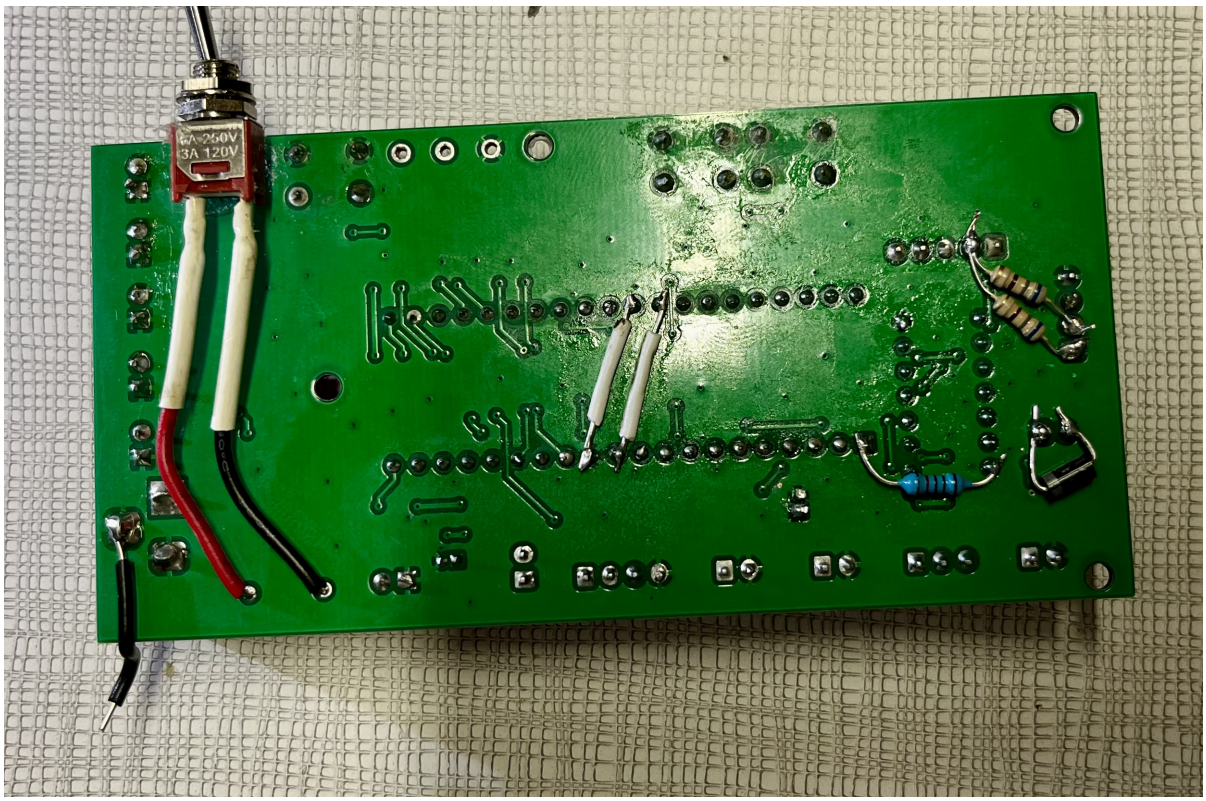


Figure 18: Back of the PCB

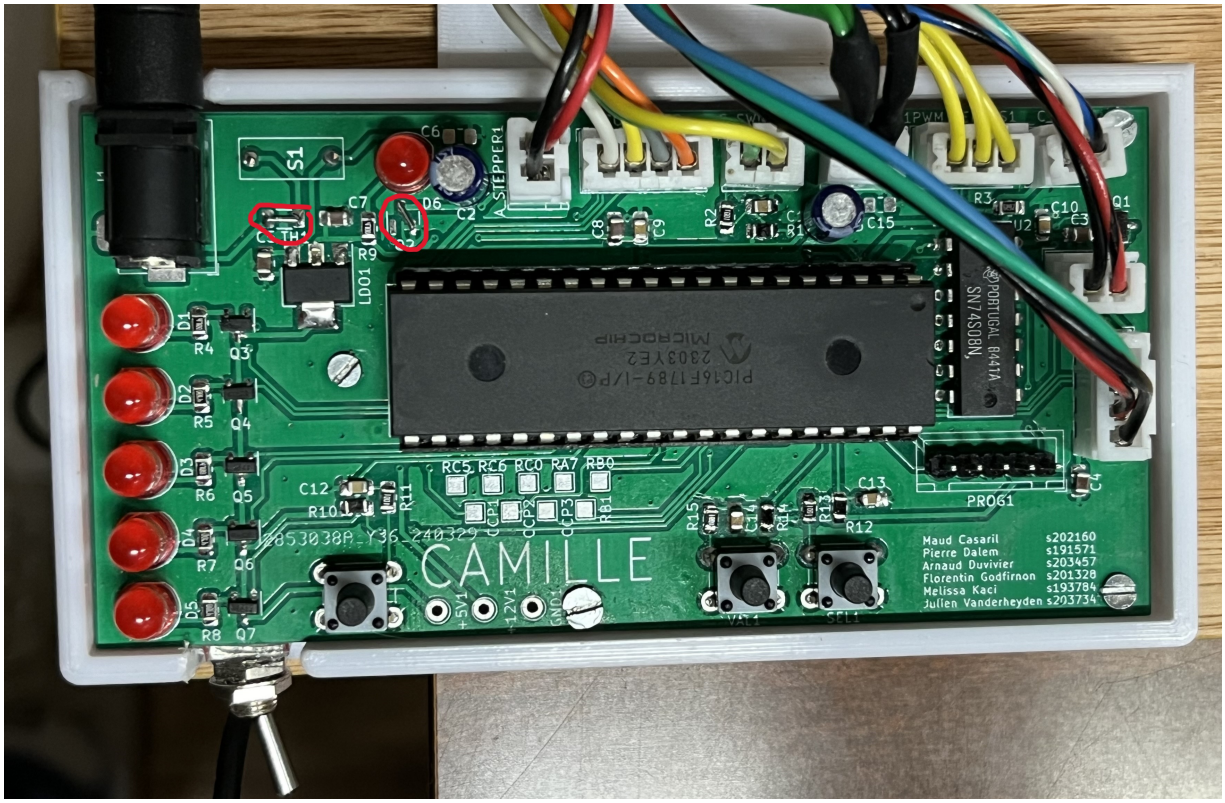


Figure 19: PCB

5 Control System

5.1 Goal of the controller

At the beginning of this project, it was envisioned to use a controller feedback loop to control the position of the platform which supports the 'Ecocup' so that the variable aspect to be controlled is the location of the glass. After some reflection, it appeared that a simple stepper motor could be used to move the platform. Due to the discrete number of steps which can be hard-coded (see section 3), the control of the position of the glass was not very interesting. In addition to this, with the constraint of coding in assembly language, a complicated controller is quite hard to implement due to the fact the various parameters have to be writable using 8 bits which limits the performance of the controller. With these constraints, some weird behaviours created by the possible low performance could lead to a wrong placement of the platform. This would mean that the glass would not be well-placed under the pods and it could damage the weight sensor or any other part of the machine which was quite a big risk.

To limit this risk, we decided to move the controller part of this project to the flow rate of water dispensing. By doing this, the weight measured by the weight sensor is used as the input of the controller and the output is a PWM signal which is sent to the DC pump to regulate its flow rate. This change of variable to control enables to move the controller aspect to a less critical part of the machine because if the glass is a little too full or not full enough regarding the goal quantity, it should not damage the machine (if it is within an acceptable margin of error). It is thus expected that while the weight measured by the weight sensor is low, the flow rate of the DC pump will be large which happens when the glass is almost empty. The more the glass is full, the slower the pumps flow rate will be. The global concept of this controller is represented in the figure 20 below.

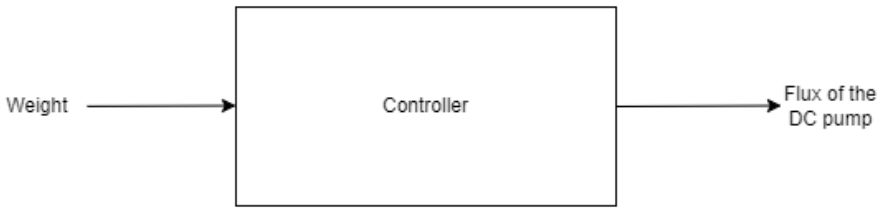


Figure 20: General idea of the controller

5.2 Input of the controller

As already mentioned, the input of the controller is the weight, which is measured by the weight sensor. After trying to control it in Arduino, it appears that the measurement is represented on 24 bits.

Firstly, it was noticed that when an object is placed on the weight sensor and left there, the last 10 bits change between each measurement returned by sensor. This means that these bits represent noise and do not have to be taken into account as their effect is not significant.

To prove this, for the same weight, the result of various measurements are given on 21 and it clearly appears that the last bits can vary for a same load (here an empty 'Ecocup' is placed on the weight sensor).

```
Value :00000000   10110100   11100110   Weight: 20.33 g
Value :00000000   10110100   01000101   Weight: 20.56 g
Value :00000000   10110100   00011011   Weight: 20.56 g
Value :00000000   10110011   11100100   Weight: 20.45 g
Value :00000000   10110100   10001110   Weight: 20.68 g
Value :00000000   10110011   00100000   Weight: 20.68 g
Value :00000000   10110011   11101110   Weight: 20.56 g
Value :00000000   10110100   01000000   Weight: 20.68 g
Value :00000000   10110100   10110010   Weight: 20.56 g
Value :00000000   10110011   01011101   Weight: 20.68 g
Value :00000000   10110011   01101010   Weight: 20.68 g
Value :00000000   10110100   10010011   Weight: 20.56 g
Value :00000000   10110100   01000111   Weight: 20.68 g
Value :00000000   10110100   00000101   Weight: 20.68 g
Value :00000000   10110010   10010110   Weight: 20.68 g
Value :00000000   10110011   11111000   Weight: 20.56 g
Value :00000000   10110011   10010010   Weight: 20.68 g
Value :00000000   10110011   10110101   Weight: 20.45 g
Value :00000000   10110011   10001011   Weight: 20.68 g
Value :00000000   10110100   00000010   Weight: 20.68 g
Value :00000000   10110011   11111110   Weight: 20.68 g
Value :00000000   10110011   10111000   Weight: 20.79 g
Value :00000000   10110100   10110111   Weight: 20.79 g
Value :00000000   10110101   01111000   Weight: 20.68 g
Value :00000000   10110100   01001100   Weight: 20.56 g
Value :00000000   10110011   01010111   Weight: 20.79 g
```

Figure 21: Measurements with an empty glass

In addition to this, after filling the glass until it is filled to the maximum, it also appears on 22 that the first 4 bits are always keep equal to 0 even in the case where the glass is fully filled. These bits do not have to be taken into account either because they will be fixed to 0 in every cases.

It is also important to notice that measurements of figure 22 represent the case where an 'Ecocup' is filled until its brim which is not a desired case. This means that when the glass is filled with 25cl (which is the filling goal), the measured weight is equal to `0000 0111 0000 0000`. The first 5 bits are set to 0, and they would only be useful if the application required measuring weights larger than this value. In this specific case only the 3 LSB (least significant bits) of the first byte can be used. But, to avoid the problem with the overflow (when the 'Ecocup' is a little bit full), it is necessary to keep the 4 LSB of the second byte.

```
Value :00001000   10000000   11001111   Weight: 249.89 g
Value :00001001   00000010   01010010   Weight: 263.85 g
Value :00001001   00000011   00010111   Weight: 263.97 g
Value :00001001   00000100   11001110   Weight: 263.85 g
Value :00001001   00000010   00100111   Weight: 263.97 g
Value :00001001   00000011   10110101   Weight: 263.85 g
Value :00001001   00000011   11101010   Weight: 263.85 g
Value :00001001   00000001   11111000   Weight: 264.08 g
Value :00001000   11111111   00001111   Weight: 264.20 g
Value :00001001   00000011   11001011   Weight: 264.08 g
Value :00001001   00000011   11001011   Weight: 264.31 g
Value :00001001   00000000   11011011   Weight: 263.97 g
Value :00001001   00000010   10001001   Weight: 264.31 g
Value :00001001   00000101   11110101   Weight: 263.97 g
Value :00001001   00000011   10111110   Weight: 264.08 g
Value :00001001   00000010   00111000   Weight: 263.74 g
Value :00001001   00000011   00110101   Weight: 264.08 g
Value :00001001   00000101   11010100   Weight: 263.85 g
Value :00001001   00000000   10001100   Weight: 264.20 g
Value :00001001   00000010   11010010   Weight: 263.85 g
Value :00001000   11111111   10010011   Weight: 264.43 g
Value :00001000   11111111   00100010   Weight: 264.31 g
Value :00001001   00000000   01101100   Weight: 264.31 g
```

Figure 22: Measurements with a glass filled to the brim

After these observations, it appears that there are 10 significant bits. It is of course better than the 24 bits that are produced but it is still too much since the input of the DC pump has to be written on 8 bits. It is thus necessary to choose 2 bits which will be removed from the signal sent to the controller. This is to ensure that the input and output are encoded with the same number of bits. To limit the effect of this operation, the two significant LSB are removed. As a result, the resolution of the measure is now of the order of magnitude of 2g because the measure can do some steps of 1.8g. This can be seen in figure 23 where a glass with just a little bit of water is weighed and only the 8 most significant bits are kept.

This error is considered acceptable since a glass a little too full or slightly not full enough compared to the filling goal is not a major problem. The parameter of the controller can be fixed while considering this precision of 1.8g without risking of encountering issues of overflowing in the glass.

```
Value :00000011   00110000   00100001                    Weight: 93.24 g
Value :00000011   00110010   10001111                    Weight: 93.24 g
Value :00000011   00110001   01100110                    Weight: 93.24 g
Value :00000011   00110001   10011011                    Weight: 95.07 g
Value :00000011   00110001   01001111                    Weight: 95.07 g
Value :00000011   00110010   00101001                    Weight: 93.24 g
Value :00000011   00110001   11011111                    Weight: 93.24 g
Value :00000011   00110000   01101110                    Weight: 93.24 g
Value :00000011   00110001   00010110                    Weight: 93.24 g
Value :00000011   00110000   10010111                    Weight: 93.24 g
Value :00000011   00101111   00110111                    Weight: 93.24 g
Value :00000011   00110001   00101011                    Weight: 95.07 g
```

Figure 23: Measurements with a glass almost empty

5.3 Output of the controller

The input which has to be sent to the DC pump is a PWM signal represented on 8 bits. The goal of this controller is to modulate the duty cycle of the PWM signal in order to control the flow rate of the DC pump as explained in the section 2.5.

5.4 Controller

It was decided to use a simple proportional controller which means that the only feedback consists in checking the error so there is only one parameter which is K_p. The global block diagram of the controller is shown in figure 24.

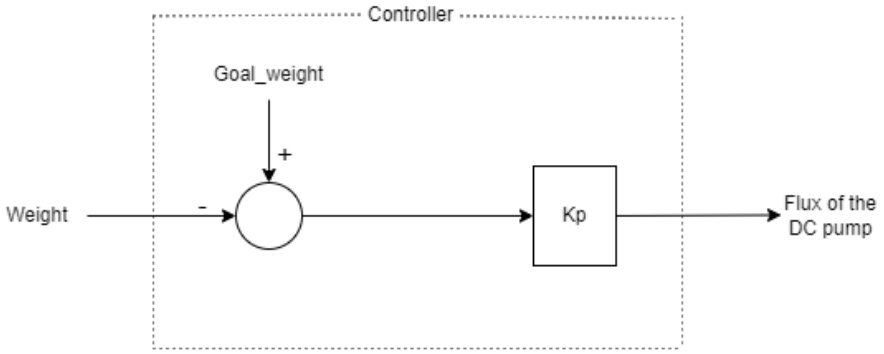


Figure 24: Block diagram of the controller

As said earlier, it can be seen that this controller has one input : the measurement on 8 bits of the weight, and one output : the PWM signal sent to the DC pump. In addition, there are two fixed constants : `goal_weight` and K_p. The constant `goal_weight` stands for the ideal weight when the 'Ecocup' is filled with 25cl written on 8 bits. The other constant K_p is the proportional coefficient of the controller.

To turn the signal of the input into an output, the first step is a difference between the `goal_weight` and the measurement from the weight sensor which gives the error:
$e(t) =$ `goal_weight`$-i(t)$ where $i(t)$ is the input at time t.

After this, the computed error is multiplied with the proportional coefficient K_p and this gives the value of the output. The resulting output is thus given by :

$$u(t) = K_p \cdot (\texttt{goal_weight} - i(t)) \tag{1}$$

This controller gives very good results since it enables the possibility of having an inversely proportional relationship between the weight and the flow rate. Indeed, at the beginning of the control phase, the 'Ecocup' is almost empty which means that a large flow rate is expected from the pump. On the other hand, when the 'Ecocup' is almost filed with the ideal quantity, it is expected that the flow rate decreases to have a smooth stop of the water filling process.

Furthermore, in the course "Linear System Design", it was explained that the K_p parameter considers the current value of the error and produces an output proportional to the difference between the desired value and the actual value. In this course, the theory of PID controllers was investigated and it was shown tha :

- K_i parameter is linked to the integral term which takes into account the previous value of the error. It considers the integral of the deviation over time and adds a correction for past errors. In practice, this guarantees zero static error. In the case of the cocktail machine, it appears after few tries that the controller always converges to the `goal_weight` value so it is not useful to add this term to the controller.
- K_d parameter is linked to the derivative term which predicts the future value of the error, thus making it possible to anticipate future changes. The derivative term is based on the speed at which the error evolves, and can therefore predict the system's future trend. This parameter is thus used to reduce the system stabilization time. In this project, the flow rate dispensed by the DC pump is expected to decrease smoothly which means that the goal here is not to have a fast decrease but more slight linear decrease. Consequently, it is useless to add the derivative effect.

Alongside this consideration of PID factors, the fact that our controller is minimal can be explained by the simplicity of the problem. Since water can only be added and it cannot be removed from the glass, oscillations are not expected. The addition of an integrative factor and/or a derivative factor is not useful especially since fast convergence is not crucial.

5.4.1 Choice of the values K_p

The value of K_p was chosen by iteratively testing and adapting this value. First, K_p was set to 1 which means that the equation of the output 1 becomes :

$$u(t) = \texttt{goal_weight} - i(t) \tag{2}$$

In this case, the output is just given by the difference between `goal_weight` and the input of the controller. At the beginning, this difference is quite large but it does not mean that the pump starts at the maximum possible PWM value. This is because the initial weight is not 0 as an 'Ecocup' has a non-null weight. This behaviour is not desired because it is expected that the flow rate begins as high as possible to fill the cup quickly at the beginning.

To try to decrease the filling time and to begin with a maximum value of flow rate, the proportional coefficient has to be increased. After trying values of K_p between 2 and 8, it was determined that the optimal trade-off between filling time and smoothness is achieved with $K_p = 3$..

5.5 Other ideas of controller

During the implementation of the controller for this machine, other controller variants were considered to enhance the resolution, which is currently set to 1.8g.

5.5.1 Controller with a table of conversion

The main thing which limits the resolution of the problem is the fact that there are 10 significant bits at the input and 8 bits are expected to be sent to the DC pump.

An idea to avoid losing too much resolution is to keep the 10 significant bits at the input of the controller. Indeed, the problem with a number written on 10 bits is that this number has to be mapped on 8 bits so it has to be divided which is a quite hard problem in assembly language. To avoid this issue, a conversion table could be used. This would enable to map all possible 10 bits input values to their corresponding 8 bits output values by hand. So the global idea of this controller is that when an input written on 10 bits is received, it is stored and the corresponding 8 bits output value will be extracted from the conversion table. It is important to note that to store the 10-bit information, two registers must be used, it is thus necessary to add some conditions to perform the mapping. The number of input values is quite large which means that the conversion table would be very long. To generate this, a python code was created in order to make the mapping between the input and the output.

With this method, all significant bits are used which means that the resolution is increased. It will thus be increased by 4 due to the use of the 2 LSB. It is also good to notice the simplicity of this method because it does not need to implement any multiplication or division which is quite hard in assembly language. Furthermore this process is quite fast and easy to understand.

On the other hand, this type of controller is well suited to a specific problem which involves 10 bits in the input and 8 bits in the output. So if we decided to use a weight sensor which gives a measurement on 9 bits for example, the conversion table becomes useless and it would have to be generated again. In addition to this, the conversion table needs $2^{10} = 1024$ entries which needs lots of lines of codes which would significantly lengthen the size of our code.

5.5.2 Controller in two phases

Another possible way to increase the resolution was considered. The global idea of this controller is to divide the controller in two phases. The switch between the two phases is decided by comparing the measurement of the weight sensor and a value M_{switch}.

- During the first phase, a feed-forward controller is used which means that the output always stays the same. The difference between the measurement and a fixed value called M_{switch} is computed to check if the measure is lower than M_{switch}. If it is, the controller stays in the first phase and if it is not, it goes in the second phase.

  The first phase corresponds thus to the first part of the filling of the glass and it is expected that the value sent to the DC pump leads to a large flow rate. It corresponds more or less to a saturation. The fixed value which is used to separate the two phases is set to 0110 0000 and the bits which are used to compute the difference are the 4 LSB of the first byte and the 4 MSB of the second byte. By doing this, the bits that are used are the same as the ones the controller used. It is good to notice that the value 0110 0000 corresponds more or less to a half filled 'Ecocup' so the first phase takes more or less the same time than the second one.

- During the second phase, a loop controller is now used. Due to the fact that the MSB bits will be fixed to 01, it is not useful to look at their values which enables to operate a switch on the bits used. Now, the bits used are the 2 LSB of the first byte and the 6 MSB of the second byte. These 8 bits will be sent in a controller which is similar to the controller used in this machine and due to the fact that the 13th and the 14th bits are used, the resolution decreases with a factor 4.

The useful bits for each phase of the controller are thus represented as :

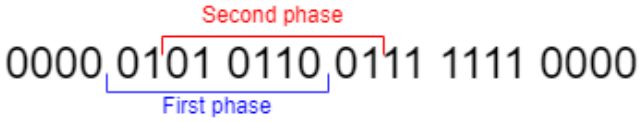


Figure 25: Representation of the different used bits for each phase

The main concept of this method is thus to separate the problem in two parts. During the first part, the controller is a simple feed-forward controller because no matter the input, the output will be hard-coded to produce a large flow rate of a flowing fluid.

During the second part, a controller using a feedback loop is implemented, which is much interesting in terms of control. However, a trade-off between these two phases has to be found. Indeed, to have an interesting controller, it is necessary to decrease M_{switch} to limit the time during when the controller is in the feed-forward phase. But, on the other hand, the value of M_{switch} enables to decrease the range of possible values which can be taken during the second phase. Indeed, during the second phase, the LSB are used and the MSB are considered fixed. The range of value that is considered in the second phase goes from M_{switch} until the maximum value defined when the 'Ecocup' is filled with 25cl ($=M_{goal}$). The problem is that the values are written on 8 bits and if this difference between M_{goal} and M_{switch} is not writable on 8 bits, an overflow can appear. The MSB of the first phase which were considered as fixed can change which is quite dangerous for the controller since it is associated with a large flow rate send by the DC pump. It would therefore be necessary to choose a trade-off when selecting the value of M_{switch}.

6 Mechanics

The whole concept of the machine requires some mechanical developments. This section will detail certain key aspects of the mechanical side of this project.

6.1 Wooden structure

Wood was chosen for the structural elements as it offers a modular approach for a relatively low price. Indeed, all parts can be cut or screwed onto the structure nearly anywhere. If however the machine were to be mass produced, a more "waterproof" material should be considered because wood can age poorly if it is in contact with too much water.

6.1.1 Main frame

The main structure used to support all the components is a wooden frame shown in figure 26. It consists of a 12mm plywood base plate. Screwed to this plate are two 3x5cm pillars. Between these two pillars is another strip of plywood. This section is used to attach all the components that need to be above the glass.

Figure 26: Image of the wooden structure of the machine

The total clearance from the base plate to the top of the structure is 42cm.

The servos along with the dispensers are screwed to the top of the machine. Once the glass is placed on the moving platform on top of the weight sensor, the distance between the dispenser and the top of the cup is 13.5cm, leaving enough space whilst still ensuring clean dispensing of the shot of alcohol.

The second component placed on the top of the structure is the pipe connecting the water supply to the cup (figure 27). The tube is guided down toward the cup with the help of a 3D printed arm giving a distance from cup to pipe of 4cm. This is however adjustable as the pipe can be manually lowered further if necessary.

Figure 27: Image of the 3D printed part guiding the water tube

Another key aspect of having such a large wooden frame is the possibility to attach components to the baseplate. Indeed, in order to have a predictable placement of our glass with regards to the different dispensing components, their relative positions need to be fixed. Therefore, both legs of the moving platform rails are screwed into the structure. This gives a very stable and repeatable positions for each cocktail configuration. Leading to an easy and consistent calibration of the stepper motor.

6.1.2 Laser cut details

In order to have a continuous presence of the brand and to improve the overall aesthetic aspect of the machine, 2 laser cut details are incorporated to the wooden frame. The first can be seen in the form of a big laser cut *Camilles* logo placed behind the moving rail. As well as an embossed cocktail machine plaque placed on the front edge.

Figure 28: Image of the laser cut parts of the machine

As well as offering a memorable bar experience to customers, branding is an important aspect. Indeed, if the goal is for clients to remember the experience and to have a familiar feeling when they see the logo, it must be present throughout the product. it must be present throughout the product. Of course, in this case, the machine is just a prototype and this is a

small detail. But branding is something that was taken into consideration if the urge to ever pursue this project further would arise.

6.2 Sliding Platform

The concept of the sliding platform was inspired by camera sliders, where the camera moves along a single axis to capture smooth video footage. Similarly, in our design, a box containing the weight sensor and the box move along one axis to position the glass accurately. To create a simple yet sturdy mechanism, two aluminum pipes are housed within 3D-printed parts. These parts were carefully designed to provide stability while accommodating certain constraints.

As the box needed to connect to a belt-driven stepper motor on one end and to a rotating piece on the other, adjustments were made to the 3D parts. A designated space for the stepper motor was added to ensure precise positioning between the two pipes, allowing for easy belt attachment. On the other end, a rotating piece was required to secure the belt without degradation. While the ideal solution would involve a support with a bearing for long-term durability, the prototype opted for a simpler approach: a stick with a timing pulley. Kits for 3D printers, featuring two timing pulleys and a belt, offered a practical solution. To ensure optimal functionality of the mechanism, proper tensioning of the belt is crucial. Two adjustments were implemented to achieve this. Firstly, on the right side, the rod holding the timing pulley can be positioned in three different slots, providing flexibility to adjust the tension as needed. Secondly, on the left side, the space allocated for the stepper motor was intentionally designed slightly larger. This extra space allows for the insertion of a small wedge to further tighten the belt, enhancing its stability and performance.

Figure 29: Timing Pulley

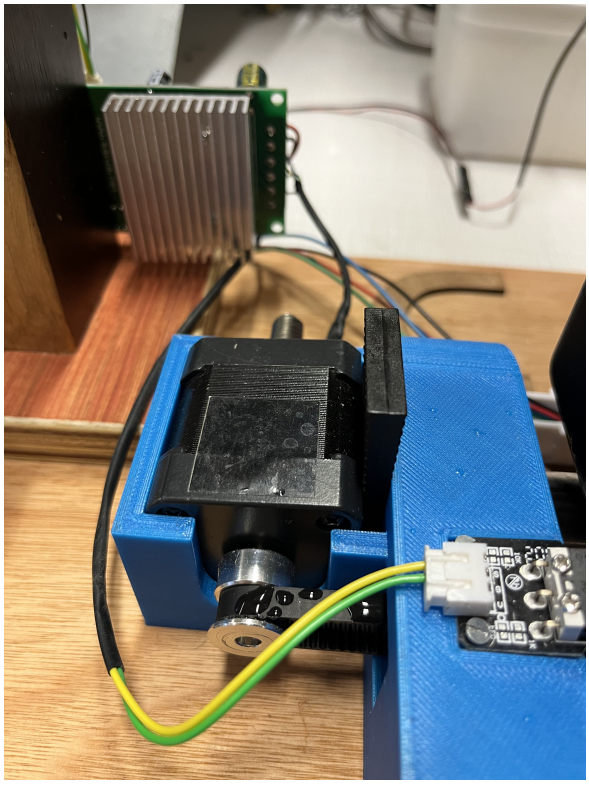


Figure 30: Wedge use for the belt tension

To address cable management concerns and to prevent disconnections during platform movement, a 3D-printed drag chain, commonly used in 3D printers, was employed. This drag chain ensures smooth movement of the platform and box while keeping wires securely in place. (see appendix A.6.2)

Figure 31: 3D printed drag chain

The design of the box prioritized practicality. It needed to be spacious enough to accommodate the weight sensor yet ensure the glass remained stationary. Thus, a lid was incorporated to center the glass without obstructing its contact with the sensor. Additionally, a hole was added to allow wires to pass through and provide access to the sensor's buttons.

6.2.1 3D printed details

To enhance the machine's appeal and leave a lasting impression, subtle details were incorporated into the 3D parts. Specifically, the phrase "Pouring joy, one drink at a time" was inscribed onto the box.

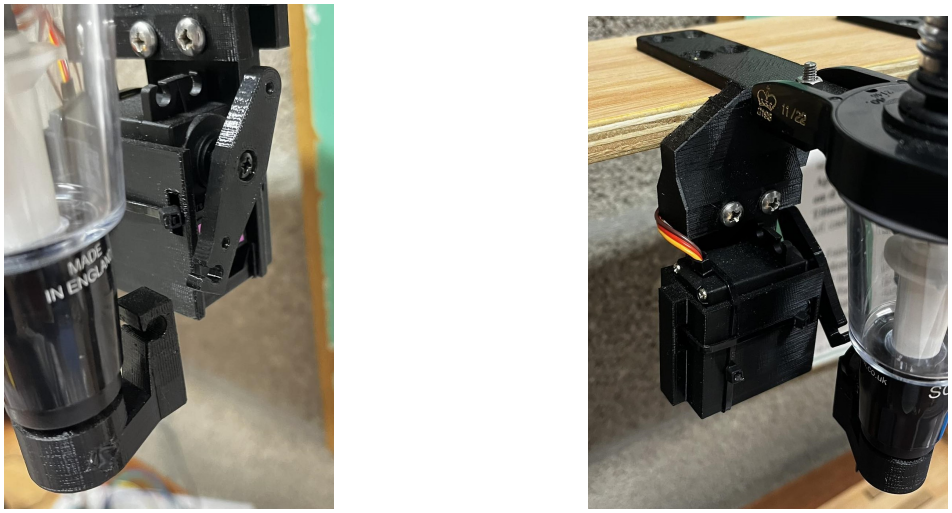
Figure 32: Sentence engraved on the box

6.3 Relation between the dispensers and the servos

As mentioned earlier, the two types of components fixed to the top of the platform are the alcohol dispensers and the water tube. The water is brought by a pump that is lower on the machine. However, dispensers are placed and actuated on the top.

The actuation is done with the help of high torque servos (specifications in section 2.1). The initial difficulty is transferring the rotational motion of the servo to the linear actuation of the dispenser. Linear actuators were considered instead of servos but were cast aside as the servos offered a smaller price for the torque capabilities as well as a very familiar operation principal.

In order to achieve this rotational to linear movement, 4 different 3D parts are used. They each had to go through several iterations to achieve the desired tolerances and strength.



(a) (b)

Figure 33: Image of all 4 parts that compose the servo-dispenser assembly

The first part is a hollow cylinder part that is placed on the bottom of the dispenser (figure 33a). This piece is glued onto the dispenser with the help of the horizontal access hole. There is also a visible hook. It is used to place an elastic band which is in turn attached to the servo arm.

The second part is a simple arm attached to the servo's shaft (figure 33a). There is another hook present to receive the elastic from the dispenser. The elastic allows for a certain amount of compliance at the start of the rotation. It also prevents breaking parts as the rotation of the servo comes with a modification of the horizontal distance between the arm and the dispenser. If both parts were linked perfectly rigidly, something could break.

The third and fourth 3D printed parts are used to fix the servo and dispenser to the structure (figure 33b). Their only imperative constraint is that they must be strong enough to withstand the actuation's of the servos. This has proven to be the case following some intensive testing. Each servo is attached with 2 bolts. A certain amount of room was left for each bolt hole on the 3D part in order to allow some fine tuning of the servos positions.

After the tests with the pads, an issue was figured out. when the servos are coming at the top of their position, as the movement of rotation is not completely transformed in a vertical displacement, the servo have difficulties to keep the pads at the high position. Therefore, the servos need more current, but the LDO can only give 800mA which is not enough for that issue. To avoid having to change the VRM and maybe the PCB, a mechanical help was added. An

elastic was used to apply a tension on the pads and therefore, the servos need less torque and so less current.

Figure 34: Mechanical help provided to the servos with the help of elastic bands

In addition to this solution, we implemented a progressive, step-by-step movement of the servo, believing it could help reduce the current consumed during its operation. However, this approach turned out to be less effective than anticipated, as it was insufficient to resolve the issue on its own.

NB: *3D renderings for all four 3D models can be found in appendix section A.6.2*

6.4 PCB placement and cable routing

The PCB is a central component as it has several buttons to control the machine. Therefore is had to be placed on an accessible point on the structure. To satisfy this constraint, it is positioned on the front left of the machine.

As mentioned in section 4, certain small things had to be modified on the PCB, one of which was the on/off switch. In order to have an easily reachable switch for a user friendly experience, it was placed below the PCB and sticking out towards the user (as can be seen in figure 19).

To have a solid placement of the PCB, a 3D printed frame was also used to fix it down securely to the structure (figure 35). This also offered an opportunity to handle cable management. A small arch is present to guide all the connectors out of the PCB and to the rest of the machine.

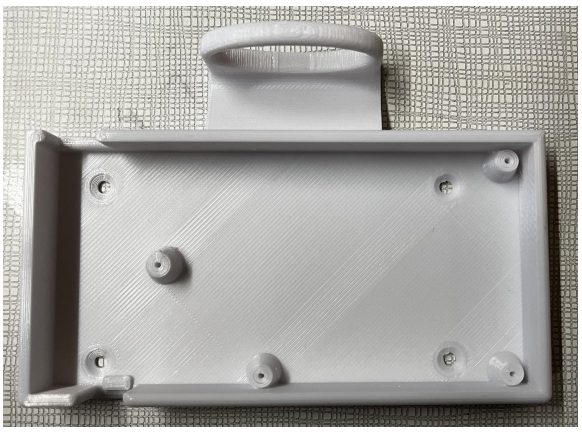


Figure 35: Image of the 3D printed PCB box

A notable component that is strategically placed is the driver module of the stepper motor. It's placement was facilitated thanks to the modular aspect of the wooden frame. In red in figure 36 it can be seen that the driver is securely attached as close as possible to the stepper motor whilst being in a position that maximizes air flow to the cooling grid of the component. As heat dissipation is a crucial aspect of this component, placement was a key consideration.

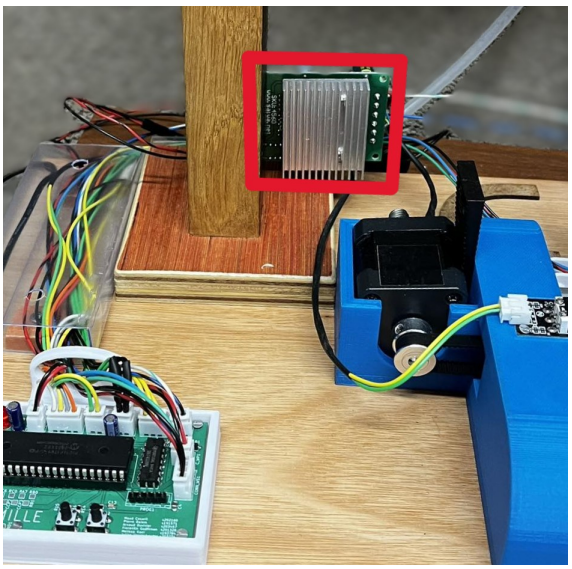
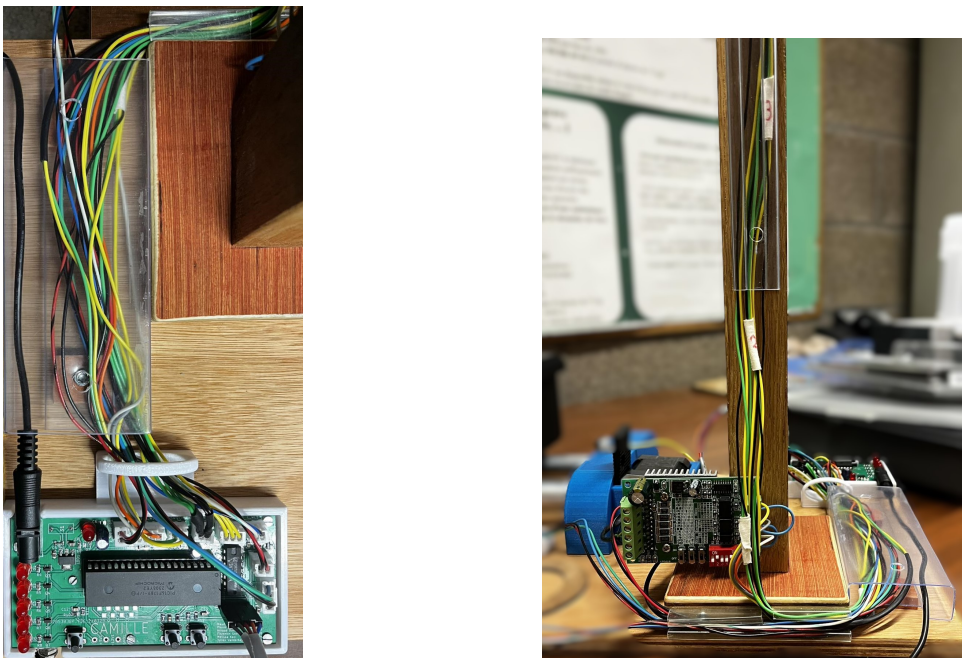


Figure 36: Image of the placement of the the stepper motor (shown in red) driver on the machine to encourage maximum air flow and cooling



(a) (b)

Figure 37: Two examples of the cable management on the machine

Cable management is key in order to identify problems and have a clean aspect. A combination of clear cable guides and heat shrink tubing was used to guide all the cables to the desired components. A white 3D printed cable chain is also used for the moving platform as mentioned in section 6.2. The clear guide is used as it gives a prototype vibe to the machine. However if the machine was to be mass produced, a more discrete and safe solution should be considered. Shielding the connectors on the PCB from any splash of liquids is an aspect that was also considered and should be addressed if the machine goes into a more advanced production phase.

7 Management and risk assessment

7.1 Project Charter

The customer asking for a solution to its problem was AEES. Discussions were held with representatives of this ASBL to define what they would need for the bar on their premises.

After discussion, the solution was to create a cocktail machine to improve the bar's flow and increase yield, while reducing customer waiting time.

In order to realize this project, several stages were defined:

- **Software** : programming a PIC16F1789 in assembly language containing all the software prerequisites defined in section 3.
- **Hardware** : an electrical circuit to couple all the components presented in the section 2.
- **Mechanics** : the cocktail machine requires a reliable mechanical structure that is also attractive, given that it will be placed on a bar for customers to see.

Tasks have therefore been defined and scheduled according to their respective stages. The precise planning is explained in the next section.

7.2 Planning

To fit in with the time constraints set by both the course and the ASBL, the project was to be carried out over 9 months, starting in October 2023. Tasks were therefore divided up within this timeframe, with the aim of completing the project by May 2024 and presenting it in June.

The distribution of time between the three different stages is shown in figure 38. The complete planning can be found in appendix A.4

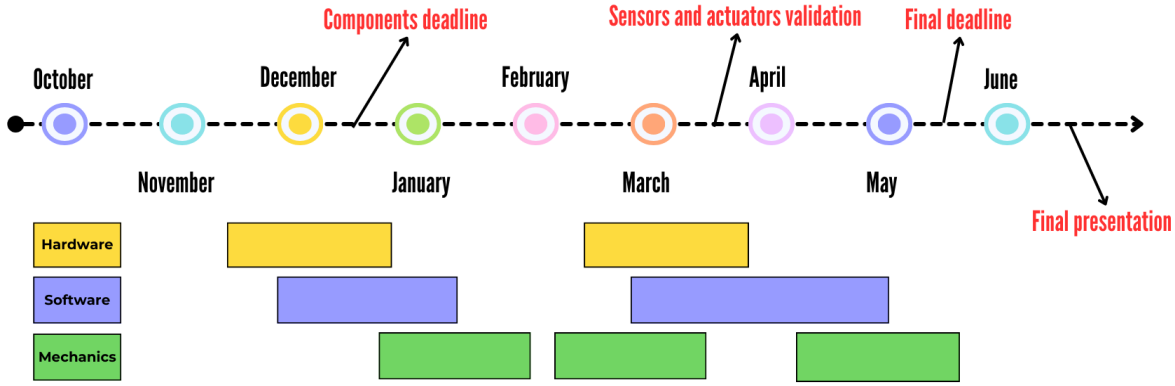


Figure 38: Planning of the project

The hardware phase began early in the year with a search for the components required for the project. In February, the selected components underwent extensive testing in Arduino and later in assembly. In March, the PCB design was completed, followed by fabrication.

At the same time, software development began in December with a first draft of a pseudocode. This pseudocode was revised and modified from February onwards, during which time the code was transcribed into assembly language. Work on the software was particularly intensive in March and April.

As for the mechanical part, this took place at different times throughout the year, according to the specific needs of the project. Several mechanical components were 3D printed and adjustments were made to the overall structure of the project, spreading the efforts over the whole year.

7.3 Acceptance Criteria

To assess the success of this project, a number of acceptability criteria have been defined.

- The machine must be able to prepare at least three different types of cocktails;
- The cocktail must be ready in less than 1 minute;
- The quantities must be accurate;
- The machine must be simple so that an average user is able to use it.

Once all these criteria have been met, the cocktail machine will produce a drinkable cocktail of your choice.

7.4 Risks assessment

A risk analysis was conducted to assess each risk of the project and to find solutions. This is an FMEA analysis.

This analysis was carried out in three stages. The first version dates from October 2023. In this version, about fifty risks were identified. Among these, about ten risks exceeded the FMEA analysis score of 100. They are explained in the following table.

Table 1: FMEA analysis - October

Component	Risk	S	O	D	RPN (S x O x D)
Weight Sensor	Humidity	7	5	6	**210**
Weight Sensor	Wrong calibration	8	4	5	**160**
Weight Sensor	Mechanical wear	6	4	5	**120**
DC pump	Mechanical wear	6	4	5	**120**
DC pump	Obstruction	7	4	5	**140**
DC pump	Corrosion	8	4	6	**192**
DC pump	Fluid leakage	8	3	5	**120**
DC pump	Overheating	8	4	4	**128**

During the month of March, this analysis was reviewed and modifications were made. The update of the previous table is found in table 2.
The explanations of the changes are :

- Weight sensor - wrong calibration : the probability and damage caused by poor calibration is the same. However it turns out that the calibration can be handled with Arduino;
- DC pump - mechanical wear : a new pump was purchased and it proved to be more manageable. After some tests, the shutdown is successful, not much risk at this level anymore;
- DC pump - Obstruction : after 3 months of tests it was possible to say that this risk was mitigated. With clean water there is no particle accumulation;

Table 2: FMEA analysis - March

Component	Risk	S	O	D	RPN (S x O x D)
Weight Sensor	Humidity	7	5	6	**210**
Weight Sensor	Wrong calibration	8	4	2	**64**
Weight Sensor	Mechanical wear	6	4	5	**120**
DC pump	Mechanical wear	6	3	3	**54**
DC pump	Obstruction	7	2	2	**28**
DC pump	Corrosion	8	3	3	**72**
DC pump	Fluid leakage	8	2	2	**32**
DC pump	Overheating	8	2	4	**64**

- DC pump - corrosion : the opportunity risk can decreased because the pump is always used with water;
- DC pump - fluid leakage : thanks to the mechanical structure, leakage is no longer a risk because the pump is in a tray and the pipe through which the water passes is well attached;
- DC pump - overheating : after three months of testing, the pump has never overheated. The probability of occurrence can be halved.

Finally, once the cocktail machine was operational, a last risk analysis was established. In this final analysis, all the points that had a score higher than 100 were reviewed and their score was reduced. The final table is the following one :

Table 3: FMEA analysis - May

Component	Risk	S	O	D	RPN (S x O x D)
Weight Sensor	Humidity	7	5	2	**70**
Weight Sensor	Wrong calibration	8	4	2	**64**
Weight Sensor	Mechanical wear	6	4	2	**48**
DC pump	Mechanical wear	6	3	3	**54**
DC pump	Obstruction	7	2	2	**28**
DC pump	Corrosion	8	3	3	**72**
DC pump	Fluid leakage	8	2	2	**32**
DC pump	Overheating	8	2	4	**64**

Finally, a last analysis was conducted in May 2024. This analysis allowed the reduction of the two RPNs that remained above 100. The explanations are as follow:

- Weight sensor - humidity : a protective box now surrounds the weight sensor and protects it from humidity;
- Weight sensor - mechanical wear : the weight sensor is now stable and placed on a platform within a box. There is no longer any risk at this level.

7.5 Budget management

At the start of the year, the Montefiore electricity department allocated a budget of 100 euros to the project. Following a search for sponsors, an additional 200 euros was granted by AEES, the student association of the Faculty.

As shown in Table 4, the budget is allocated across several categories.

Table 4: Detailed budget allocation

Expense section	Expense detail	Price [€]
Hardware	Weight sensor	23.16
	Water level sensor	8.61
	DC pump	2.74
	Power supply	15.22
	Servo	16.99
Mechanics	Dispenser pods	28.2
	Bottles	13.68
	Rail system	11
	Pointed screw	3.79
	Hose, seals and JST	10.28
	Belt	8.48
Alcool	Pastis/Peket/Get27	52.45
	Total:	**194.6**

In total, less than 200 euros of budget were used to complete the whole project.

8 Improvements

8.1 What could have been done differently

In this section, we will review several decisions that were made and explain how they would be carried out differently if the project were done again.

- The choice of the servo motor: As the chosen servos have rotational movement and not linear, a complex conversion of the movement is needed to enable the pods to be pull up. This work could have been avoided if linear actuators were used instead. Moreover, as the movement is not perfectly along the same axis, some torque is applied horizontally and not just vertically as wanted. This has caused some issues, as the servos can drive a lot of current when they are in a static position.

- Make the power supply system more robust: The choice of the PCB components was not studied enough, which lead to little issues and limitations as the output current of the LDO, which is 800mA. Coupled with a stall current of 2.5 A for the servos, this is not a something that is wanted.

8.2 Further improvements

As the code had to be done in assembly, the possible improvements are complex. Something that might seem easy could take a lot of work to put in place. Here are some ideas of improvements that would have been great:

- From a protective standpoint, since we are working with water around electronic circuits, introducing some protection to collect any water that overflows from the glass would be a great idea. This would make the machine more robust against errors. Having a closed bucket for the water and pump would also be better.

- From an aesthetic standpoint, hiding all the wires, the driver of the stepper, the stepper and the servos to leave only the mechanical parts visible would improve the appearance of the machine. Some decorative LEDs would also increase the aesthetic of the machine.

- Regarding the machine's functionalities, adding a mechanism to mix the cocktail and incorporate sparkling liquids would elevate the machine to another level. However, as previously mentioned, this would have made the assembly process significantly more complex.

- To improve ease of use and enhance the user experience, a small LCD screen could be added. This would make the machine simpler and more intuitive to operate. Additionally, to make the machine easier to use, a more stable holder for larger alcohol bottles could be designed, allowing users to place the actual "standard" bottles directly on the machine.

9 Conclusion

After nine months working on this project, our team really enjoyed the journey of developing this cocktail machine. Working on a tangible project from the choice of the components to the assembling was really rewarding. Furthermore, it enabled us to gain skills in terms of teamwork, problem-solving, and project management. A small downside was receiving the components only in the second half of the academic year which hindered the possibility to physically work on the project for a longer period.

Despite some frustrations caused by the constraint of coding in assembly language, it allowed us to truly understand how each component worked and the impact of the software on the hardware components. It also enabled us to combine technical skills with practical application. We are proud of the functional cocktail machine we created, to have been able to meet all the acceptance criteria and even to have done better since the cocktail is made in 30 seconds!

List of Figures

1 First schematic of the machine . . . 1
2 Photo of the completed cocktail machine . . . 1
3 Illustration of the servo motor's duty cycle and PWM along with the pin assignment 2
4 Diagram of the water level sensor . . . 3
5 Evolution of the resistance in the electrodes of the sensor as a function of their height with respect to the surface of the water . . . 4
6 Stepper motor driver pins . . . 5
7 Connections between the connector to the stepper motor and the PIC . . . 6
8 Diagram of the 3 possible configurations for the stop switch . . . 6
9 Stop switch seen from above . . . 6
10 Oscilloscope Traces of the "bug" making the PIC16 reset . . . 7
11 DC pump waveform when a freewheeling diode has been added . . . 7
12 Weight sensor when all parts are assembled . . . 8
13 Oscilloscope trace of an I2C communication . . . 9
14 Binary multiplication . . . 15
15 Schmitt Trigger diagram . . . 16
16 Electrical Schematic . . . 19
17 Layout of the Final PCB . . . 20
18 Back of the PCB . . . 22
19 PCB . . . 22
20 General idea of the controller . . . 23
21 Measurements with an empty glass . . . 24
22 Measurements with a glass filled to the brim . . . 24
23 Measurements with a glass almost empty . . . 25
24 Block diagram of the controller . . . 25
25 Representation of the different used bits for each phase . . . 28
26 Image of the wooden structure of the machine . . . 29
27 Image of the 3D printed part guiding the water tube . . . 30
28 Image of the laser cut parts of the machine . . . 30
29 Timing Pulley . . . 31
30 Wedge use for the belt tension . . . 31
31 3D printed drag chain . . . 32
32 Sentence engraved on the box . . . 32
33 Image of all 4 parts that compose the servo-dispenser assembly . . . 33
34 Mechanical help provided to the servos with the help of elastic bands . . . 34
35 Image of the 3D printed PCB box . . . 35
36 Image of the placement of the the stepper motor (shown in red) driver on the machine to encourage maximum air flow and cooling . . . 35
37 Two examples of the cable management on the machine . . . 36
38 Planning of the project . . . 37
39 Renderings of the four different 3D parts explained in section 6.3 . . . 44
40 Renderings of the four different 3D parts explained in section 6.3 . . . 45

A Appendix

A.1 Component data-sheets:

- Servo motor
- Water level sensor
- Stepper motor
- Stepper motor driver module
- DC Pump
- Weight Sensor

Click on the component name, it acts as a hyperlink

A.2 PCB components data-sheets:

- PIC
- VRM
- Transistor
- AND Gates
- Power supply

A.3 Codes:

- Arduino HX711 I2C library
- **Assembly code**

A.4 Detail planning

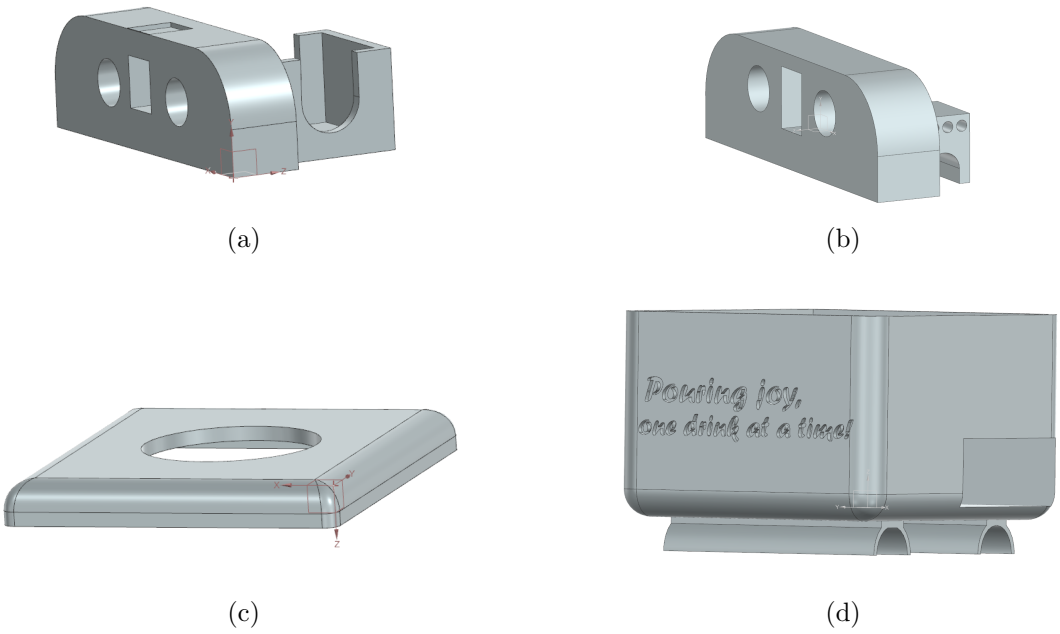
Planning

A.5 Video

A demonstration video can be found here: link

A.6 Renderings of 3D parts

A.6.1 Renders from section 6.2

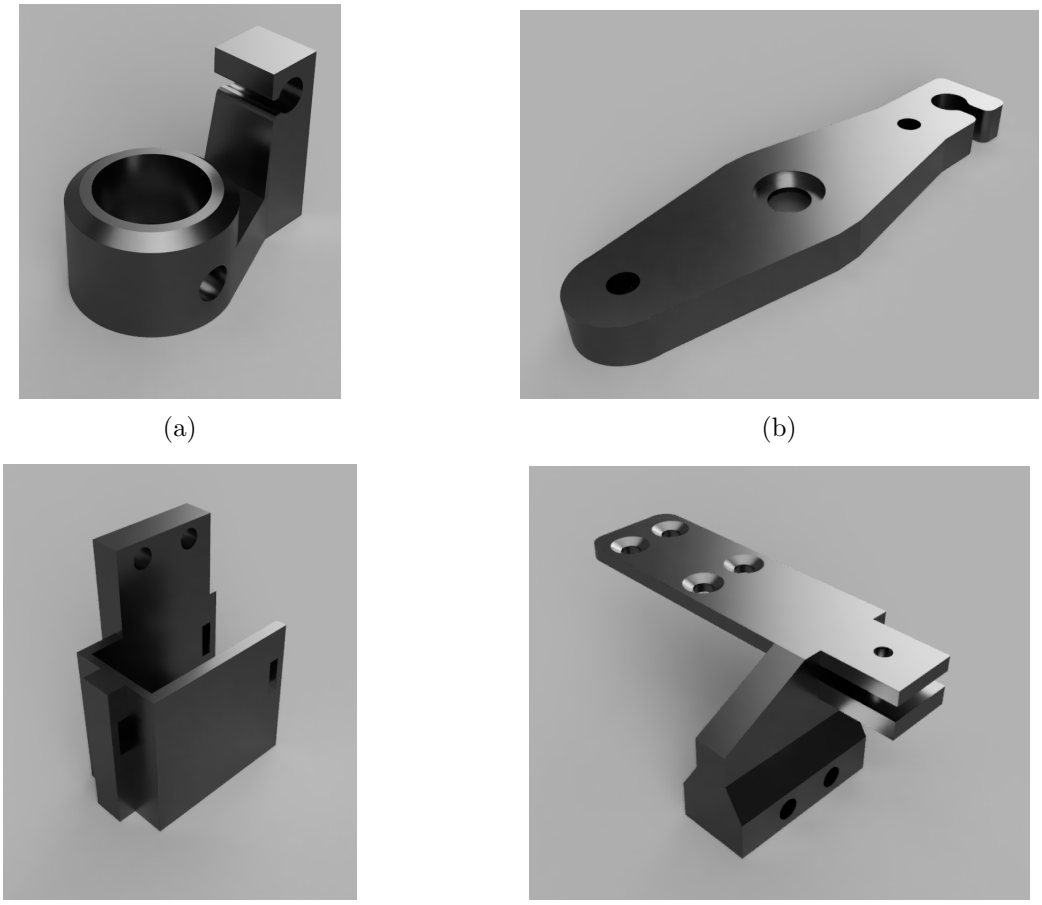


(a) (b)

(c) (d)

Figure 39: Renderings of the four different 3D parts explained in section 6.3

A.6.2 Renders from section 6.3



(a) (b)

(c) (d)

Figure 40: Renderings of the four different 3D parts explained in section 6.3